

CSCI E-25

Introduction to Deep Neural Networks

Steve Elston

1.0 Overview

This lesson introduces you to the basics of neural network architecture in the form of deep forward networks. This architecture is the quintessential deep neural net architecture. In this lesson you will master the following:

- Why is deep learning important and how it relates to representation, learning and inference.
- How a basic Preceptron works.
- How to apply different types of loss functions.
- Understand why nonlinear activation is important and why rectified linear units are a good choice.
- How back propagation works, and how you apply the chain rule of calculus to determine gradient.
- Understand the architectural trade-off between depth and width in deep networks.
- Know how and why you must apply regularization to deep neural networks.
- The importance of learning rate in training deep neural networks.

1.1 Why is deep learning important?

Deep learning methods are a form of **artificial intelligence (AI)** or **machine intelligence**. More specifically, deep learning algorithms are a type of **machine learning**.

What properties does machine intelligence require? There have been many answers to this question over the history of computing. In this case, we will take a practical view, sometimes known as **weak AI**. There are three key properties an intelligent machine must have. Deep learning algorithms are one of a few classes of algorithms that

can do the following, essential to machine intelligence:

1. **Representation:** An intelligent machine must be able to represent a model of the world it interacts with in a general manner. Representation is key to intelligence. Without a good representation the best learning and inference algorithms will struggle. Whereas, good representation can greatly facilitate learning and inference. In conventional machine learning the representation is model and a set of features. The representation is limited to what the features can provide directly. Deep learning algorithms, on the other hand, learn learn complex representations from raw features. This behavior allows deep learning algorithms to approximate complex relationships. Further, the representations learned often generalize well, up to a point.
2. **Learning:** As you likely guessed from the very name, deep learning algorithms learn from data. Whereas, conventional machine learning is focused on inference, deep learning algorithms learn both inference and representations. As a result, deep leaning algorithms are more complex and therefore harder to train than conventional machine learning algorithms.
3. **Inference:** Any machine intelligence algorithm must be able to perform inference. The inference is the result produced given new input data. To be useful, the inferences produced by a machine intelligence algorithm must **generalize** beyond the cases used for learning or training. Good generalization requires both good representations and learning which can deal with the complexity of diverse situations. Some deep learning algorithms can approach human levels of performance in inference tasks such as recognizing objects in images or understanding natural speech.

The figure below shows a highly abstracted view of machine intelligence, showing the relationship between representation, learning and inference. In simple terms, the representation is learned and then used to make inferences. Errors in the inferences can be used to improve the learning of the representation.

Drawing



Schematic for creating machine intelligence

That's it! The entire rest of this course will focus on just these three points: representation, learning and inference!

1.2 Installing Keras

This notebook will provides a first look at using the Keras package to define, train and evaluate deep learning models with Keras. The Keras package is a wrapper on TensorFlow, intended to abstract and simplify the definition, training and execution of TensorFlow deep learning models. You can find extensive well-written documentation for Keras [here](#). The book [Deep Learning with Python](#) by François Chollet, the creator of Keras, provides in-depth examples and discussion on a wide range of deep learning applications.

By the end of this lesson you will be able to work with basic feedforward architecture multi-layer neural nets. Feedforward networks are one of a class of basic models called **sequential models** which are easy to define with Keras. Some basic regularization is introduced. Additional regularization methods are covered in a subsequent lesson.

Keras is part of the base package, as of the release of TensorFlow 2. Before proceeding make sure you have TensorFlow 2 installed in your environment. [Follow these instructions..](#)

Note: As an alternative to working with a local installation, you may choose to use the [Google Colabrotory](#). The Colabrotory virtual environment includes Anaconda, TensorFlow and Keras. However, the use of shared resources can result in slow execution. Generally a paid account, Colab Pro, or Pro+ works considerably better.

Note: This notebook was constructed and tested using Anaconda 3 with Python 3. It is assumed that the standard Anaconda stack has been installed.

```
In [1]: !pip install keras
```

Requirement already satisfied: keras in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (3.8.0)

Requirement already satisfied: absl-py in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from keras) (2.1.0)

Requirement already satisfied: numpy in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from keras) (1.26.4)

Requirement already satisfied: rich in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from keras) (13.9.4)

Requirement already satisfied: namex in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from keras) (0.0.8)

Requirement already satisfied: h5py in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from keras) (3.13.0)

Requirement already satisfied: optree in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from keras) (0.14.0)

Requirement already satisfied: ml-dtypes in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from keras) (0.3.2)

Requirement already satisfied: packaging in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from keras) (24.2)

Requirement already satisfied: typing-extensions>=4.5.0 in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from optree->keras) (4.12.2)

Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from rich->keras) (3.0.0)

Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from rich->keras) (2.19.1)

Requirement already satisfied: mdurl~=0.1 in /usr/local/anaconda3/envs/e25/lib/python3.11/site-packages (from markdown-it-py>=2.2.0->rich->keras) (0.1.2)

```
In [2]: from tensorflow import keras
        from tensorflow.keras import datasets
```

2025-03-02 21:11:19.617132: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: AVX2 AVX512F AVX512_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.

```
In [3]: from keras.api.datasets import mnist
        import tensorflow as tf
        import keras.api.utils as ku
        import keras.api.models as models
```

```
import keras.api.layers as layers
from keras.api.optimizers import RMSprop
from keras.api.optimizers.schedules import ExponentialDecay
from keras import regularizers
import numpy as np
from math import exp
import matplotlib.pyplot as plt
import seaborn as sns

%matplotlib inline
```

```
In [4]: tf.__version__
```

```
Out[4]: '2.16.2'
```

Skip

2.0 Forward propagation: The representation problem

To create useful neural network we need a **representation** that has two important properties.

First, there needs to be a way to represent complex functions of the input. Without this property, nothing is gained, since there are numerous machine learning algorithms that work with simple representations. We will spend the rest of this section exploring this problem.

Second, the representation needs to be learnable. Quite obviously, no machine intelligence representation is useful if there is not a practical algorithm to learn it. We will take up this problem in another section.

2.1 Linear networks

Let's start with the simplest possible network. It has inputs, and an output. The output is a **afine transformation** of the input values. We say this network performs an afine transformation since there is a bias term b .

Drawing



****Figure 2.1**** ****A simple affine network****

This output y of this network is just:

$$y = f(x) = \sum_i w_i \cdot x_i + b$$

This network performs linear regression. Being able to perform only affine transformations, it can't do anything else.

This representation is certainly learnable. However, it does not gain us anything over familiar linear regression methods.

2.2 The perceptron

To get started, let's have a look at a simple **perceptron** model. The perceptron was proposed by Rosenblatt (1962). He built on the earlier attempts at a neural network models by McCulloch and Pitts (1943) and Hebb (1949). The perceptron adds **nonlinear activation** to the affine network.

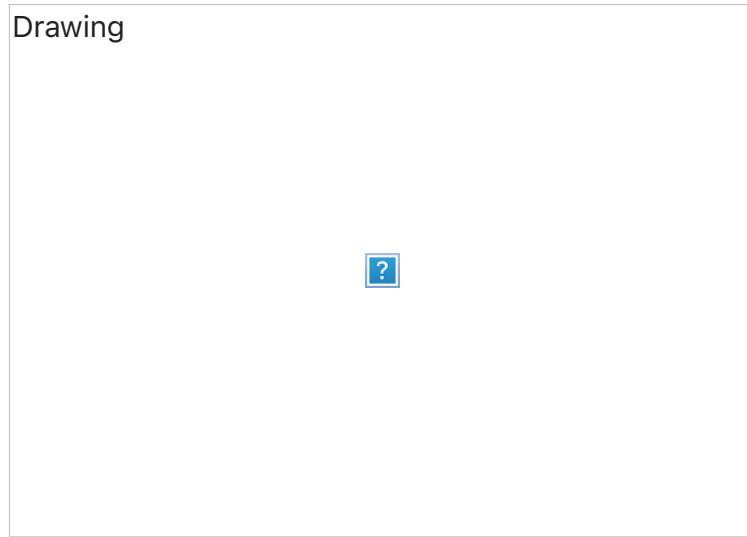


Figure 2.2 Schematic of perceptron with nonlinear activation

The output y of the perceptron is given by the following:

$$y = f(x) = \sigma\left(\sum_i w_i \cdot x_i + b\right)$$

The output of the network is now nonlinear, give the **activation function** $\sigma(x)$.

But, the perceptron is nothing more than a logistic regression classifier. The fact that the perceptron could only solve linearly separable problems was pointed out by Minsky and Papert (1969). The failure of the perceptron to learn an **exclusive or (XOR)** function is well known. See for example, Section 6.1 in GBC.

Again, this representation is certainly learnable. However, as before, it does not gain us anything over well known logistic regression models.

2.3 Forward networks - We're gonna need a better representation!

The problem with the perceptron is one of representations. There is no way that this simple network can represent anything but a linearly separable function. To represent more complex functions, we need a more complex network. In more technical terms we need a network with greater **model capacity**.

What we need is a network with layers of **hidden nodes**. The figure below shows a simple example of a neural network with one **hidden layer** with two nodes. Since every node (including inputs) is connected to every other node we call this architecture a **fully connected neural network**.

Drawing



****Figure 2.3 Fully connected neural network with single hidden layer****

Let's walk through some aspects of these diagrams.

1. The neural network is divided into three layers. The input layer, the hidden layer and the output layer.
2. The values in the input layer are multiplied by a weight matrix, W^1 .
3. The nodes in the hidden layer sum their inputs and add a bias term, b^1 .
4. The outputs of the hidden layer nodes are multiplied by a weight vector, W^2 .
5. The output layer sums the inputs and adds another bias term, b^2 .

2.4 Neural network architectures - Finding representations

The representations achievable by neural network with just a single hidden layer are quite powerful. In fact, Cybenko (1989) showed that such a network with an infinite number of hidden units using sigmoidal activation can approximate any arbitrary function. Hornik (1991) generalized this to apply to any activation function. We call this theorem the **universal approximation theorem**.

A universal approximation theorem may see like a really exciting development; especially if you are a machine intelligence nerd. However, one must be circumspect when viewing such a result. A representation with an infinite number of nodes cannot be learned in any practical sense. Still it is comforting to know that, at least in principle, a representation can be learned for arbitrarily complex problems.

While infinitely wide networks with a single layer are unrealistic, we are not limited to one dimension. In fact, depth is typically more effective at creating complex representations rather than width in neural networks. Depth is measured by the count of hidden layers stacked one on top of the other in the network. Hence, the term deep neural networks.

The Figure 2.4 below shows the results of an empirical study by Goodfellow, Shlens and Szegedy (2014) of accuracy of the network vs depth. Notice that accuracy increases rapidly with depth until about 8 layers, after which the effect is reduced.

Drawing



Figure 2.4 Empirical results of accuracy vs. number of layers Diagram from Goodfellow et. al. 2014

Another view of the empirical study by Goodfellow et. al. is shown in Figure 2.5 below. In this case accuracy verses number of model parameters is compared for three different network architectures. The deeper network (11 layers) makes more efficient use of the parameters in terms of improved accuracy. The number of parameters in a layer is approximately the total number of parameters divided by the number of layers. Notice that for the particular case tested convolutional neural networks are more efficient than fully-connected networks. We will discuss convolutional neural networks in a subsequent lesson.

Of particular interest is the fact that the fully-connected network and the shallow convolutional neural network appear to be over-fitting as the test accuracy actually decreases as the number of parameters increases. We will discuss the significant problems of over-fitting in neural networks in a subsequent lesson.

Drawing



****Figure 2.5 Empirical results of accuracy for different network architectures**** Diagram from Goodfellow et. al. 2014

Summary: Deep networks tend to produce better models, with less tendency to over-fit, for a given level of complexity.

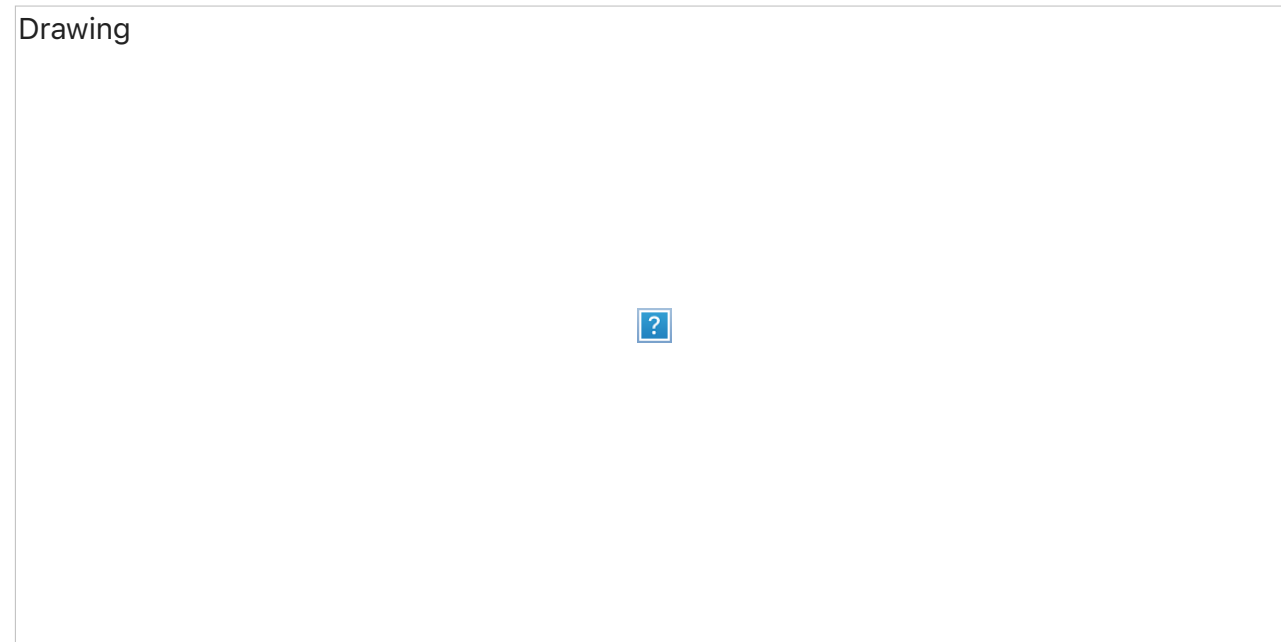
2.5 Computational graphs

There is another way to look at neural nets, computational graphs. A computational graph breaks down the steps of a complex algorithm into steps.

Computational graphs provide a way to organize complex computations in an efficient manner. Widely used computational frameworks such as Tensor Flow, CNTK, and Torch all use computational graphs. Organizing computations in a graph allows these platform to minimize memory transfers. In simple terms, the platform can look ahead in the graph and organize data and computational results so as to minimize memory transfers. As a result, such platforms can be significantly faster than, say, Python Numpy. Systems like Numpy require memory transfer

before each operation, which typically take more time than the actual computation.

The diagram below decomposes the single hidden layer neural network discussed in the previous section into a computational graph.



****Figure 2.6 Computational graph for fully connected neural network of Figure 2.3****

Let's walk through this graph, step by step.

1. The $n \times 2$ weight tensor, W^1 , is multiplied by the 1-dimensional input tensor $x \in \mathbb{R}^n$, giving the result $U^1 \in \mathbb{R}^2$.
2. The 1-dimensional bias tensor $b^1 \in \mathbb{R}^2$ is added to U^1 , giving $U^2 \in \mathbb{R}^2$.
3. The activation function $\sigma_h(x)$ is applied to U^2 , producing $U^3 \in \mathbb{R}^2$.
4. The dot product between the weight tensor, $W^2 \in \mathbb{R}^2$ and U^3 is computed giving $U^4 \in \mathbb{R}^1$.
5. The bias, $b^2 \in \mathbb{R}^1$ is added to U^4 giving $U^5 \in \mathbb{R}^1$.
6. The output activation function $\sigma_o(x)$ is applied to U^5 giving the output $Y \in \mathbb{R}^1$.

As you can see, the computational graph provides a complete specification for the single hidden layer neural

network.

2.6 Activation functions

Without a nonlinear activation function, a neural net is just an affine transformation. Affine transformations limit representation to only linearly separable functions. To create more general representations **nonlinear activation functions** are required.

In present practice, four types of activation functions are generally used for fully connected networks.

1. **Linear** activation is used for the output layer of regression neural networks.
2. The **rectilinear** activation function is used for most hidden units. The rectilinear activation function is often referred to as **ReLU**.
3. A **leaky rectilinear** activation acts like a ReLU function for positive inputs, but has a small negative bias or leakage for negative input values. The leaky ReLU activation function can improve training for some deep neural networks.
4. The **logistic** or **sigmoid** activation function is used for binary classifiers.
5. The **softmax** activation function is used for multi-class classifiers.

Rectilinear functions are typically used as the activation function for hidden units in neural networks. The rectilinear function is defined at:

$$f(x) = \max(0, x)$$

The rectilinear function is linear for positive responses and zero for responses less than 0.0. Notice that the derivatives of the rectilinear function are not continuous. While this might seem to be a problem, in practice, even gradient-based optimization functions work well with this activation function.

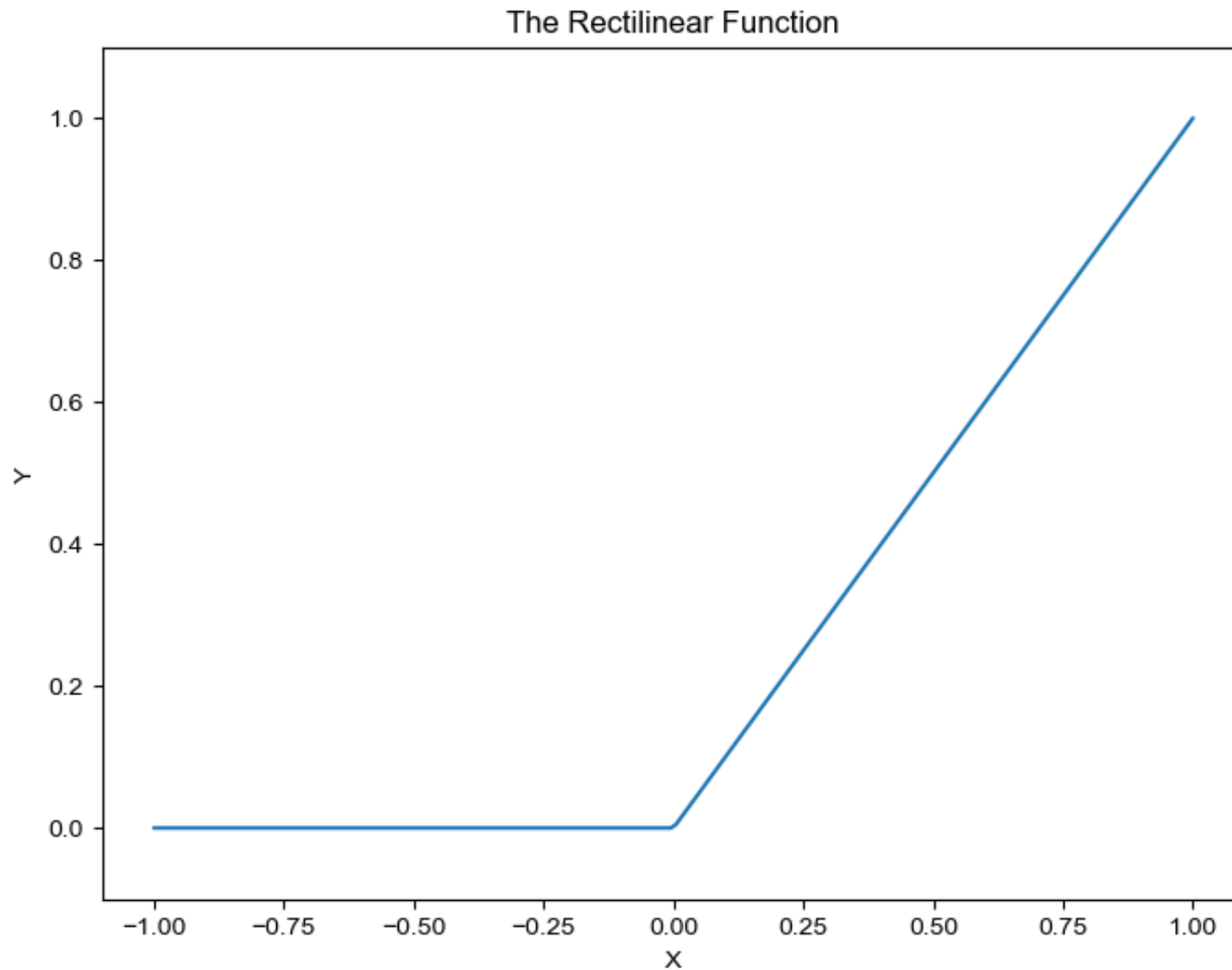
The rectilinear function is plotted in the cell below:

```
In [5]: def reclu(x): return(max(0,x))

def plot_figs(x,y,title, figsize = (8, 6)):
```

```
plt.figure(figsize=figsize).gca() # define axis
sns.set_style("darkgrid")
plt.plot(x, y)
plt.ylim((-0.1,1.1))
plt.title(title)
plt.xlabel('X')
plt.ylabel('Y')

x = np.linspace(-1.0, 1.0, 200)
y = [reclu(y) for y in x]
plot_figs(x,y,'The Rectilinear Function')
```



Another widely used activation function is the **logistic** or **sigmoid**. The sigmoid is used as the activation for the output layer of a binary classifier. The general sigmoid function can be written as:

$$\sigma(x) = \frac{L}{1 + e^{-k(x_0 - x)}}$$

where $L = \text{max value}$, $k = \text{slope}$, $x_0 = \text{sigmoid midpoint}$

With $L=1$, $k=1$, and $x_0 = 0$, the logistic function becomes:

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{1+e^x}$$

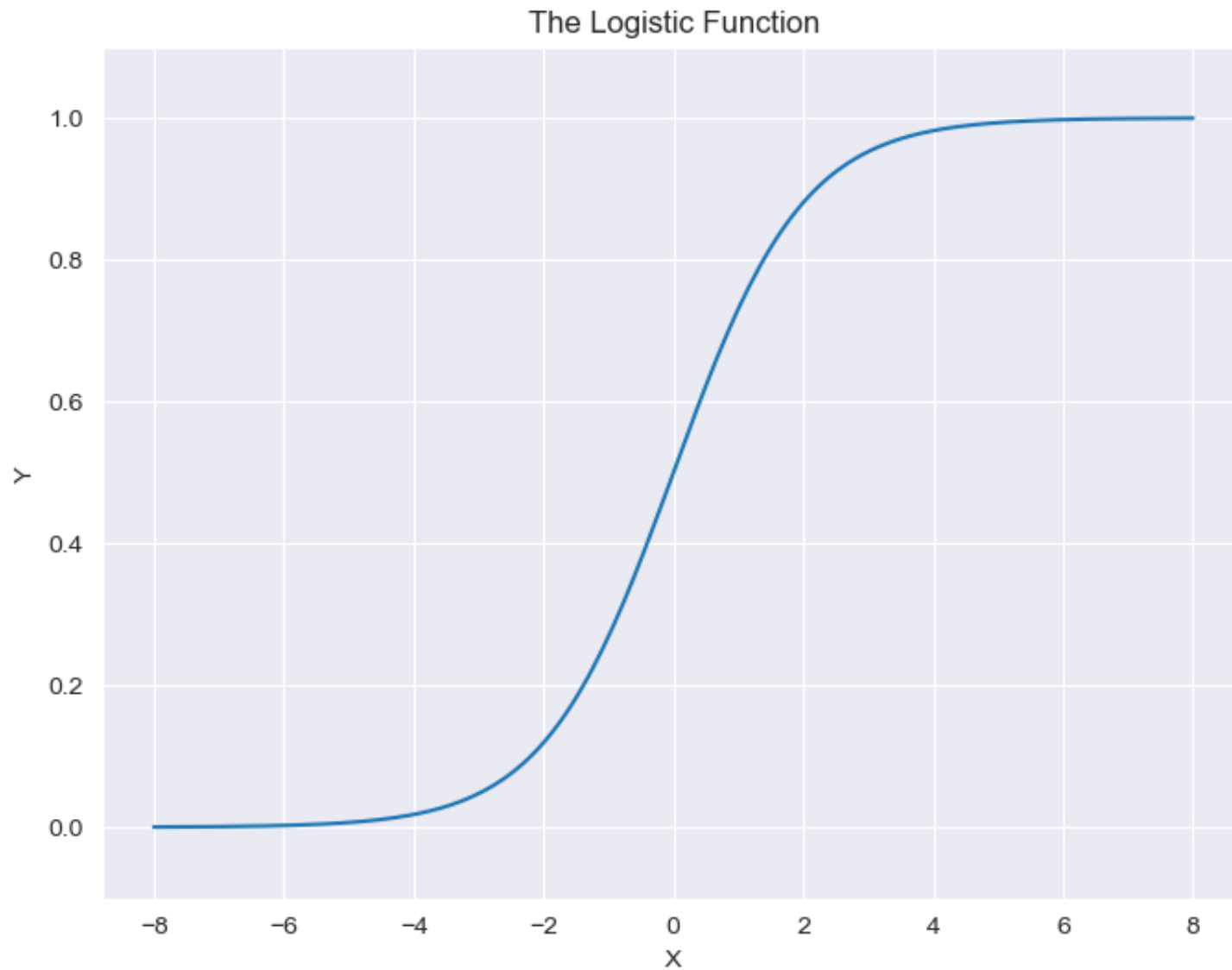
The sigmoid function can asymptotically approach 0 or 1, but will never reach these extreme values. However, because of the rapid decrease in the derivative away from 0 the sigmoid can **saturate** when using gradient-based training. For this reason, the sigmoid is typically not used for hidden layers in neural networks.

When used in a the binary classifier a threshold is set to determine if the result is 0 or 1. The threshold can be adjusted to bias the result as desired.

The code in the cell below plots the sigmoid function.

```
In [6]: def sigmoid(x): return exp(x)/(1 + exp(x))

x = np.linspace(-8.0, 8.0, 200)
y = [sigmoid(y) for y in x]
plot_figs(x,y,'The Logistic Function') #, figsize = (5,3))
```



The **softmax** function or **normalized exponential function** is used for the output activation function of a multi-class classifiers. The softmax function is the multinomial generalization of the sigmoid or logistic function. The probability of each class j is written as:

$$\sigma(z_j) = \frac{e^{z_j}}{\sum_{k=1}^K e^{z_k}}$$

The normalization $\sum_{k=1}^K e^{z_k}$ ensures the sum of probabilities for all classes add to \$1.0\$. The class selected by the classifier is the class with the largest value of $\sigma(z_j)$.

2.7 Computational example

Now that we have gone through some basic theory for feed-forward networks, it's time try a simple example. You will construct a fully connected network to compute this simple function:

$$y = x_1 - x_2$$

Comment. You have likely have noticed that this function is linear and can be computed easily without a neural network. Of course, that is not the point. We use a simple function to make the results easy to understand.

Exercise 5-1: You will create and test a simple neural network implemented using matrix multiplication with Numpy. The architecture of the neural network is similar to the one shown in Figure 2.3, with 2 input units, 2 hidden units and 1 output unit. There are a total of 6 weights in two tensors. The neural network for this example does not require any bias terms.

1. As a first step, create test data for 3 possibilities; $x_1 > x_2$, $x_1 = x_2$, and $x_1 < x_2$, and with positive and negative values, or $x = [(2,1), (1,1), (1,2), (0,0), (2,-1), (-1,-1), (-2,1), (-1,-2)]$ as the input tuples.
2. Directly compute and print the evaluation of the function, $y = x_1 - x_2$, for each tuple.

Note: The network you are asked to construct is simple and all weights must in the set $\{-1, 1\}$. You can take advantage of the symmetry of the function you must approximate to determine these weights by inspection. If you wish, you will find it easy to compute the partial derivatives of the function to be approximated. However, this is not necessary if you carefully inspect the network and consider the responses required.

In [7]: `## Your code goes here`

```
x = [(2,1), (1,1), (1,2), (0,0), (2,-1), (-1,-1), (-2,1), (-1,-2)]
for x_in in x:
    print(x_in[0] - x_in[1])
```

```
1
0
-1
0
3
0
-3
1
```

- Now that you have the test data you can move to the next step. Determine the values of the 2×2 weight tensor between the input layer and the hidden layer (input tensor) and the 2×1 weight tensor between the hidden and output layer (output tensor). In the code cells below create as Numpy arrays and print these tensors. *Hint:* Keep in mind that the input tensor must be symmetric, with correct signs on the $\{-1,1\}$ weights. Likewise, the output tensor $\{-1,1\}$ weights must have the opposite signs.

```
In [8]: ## Your code goes here
W_1 = np.array([[1.0, -1.0], [-1.0, 1.0]])
print(W_1)

[[ 1. -1.]
 [-1.  1.]]
```

```
In [9]: ## Your code goes here
W_2 = np.array([1, -1])
print(W_2)

[ 1 -1]
```

- Now, it is time to compute the results and check them. To create the computational process follow the graph in Figure 2.6, but ignoring the bias terms, b^1 and b^2 . Create a function, `hidden`, to compute the output of the hidden layer using the formulation with **rectalinear**

activation, δ :

$h = \delta(W^1 \cdot x)$ Create a second function, **output**, to compute the vector product of the weight vector with the output vector of the hidden layer using a **linear activation**:

$o = W^2 \cdot h$

5. Execute the two functions while iterating over the input tuples and verify the output is correct. If not, reconsider the values of your weight tensors.

```
In [10]: ## Your code goes here
def hidden(x, W):
    """Computes the output of the hidden layer"""
    h = np.dot(W, x) # product of weights and input vector
    return np.array([recl(x) for x in h]) # apply activation function and return

def output(h, W):
    """Computes the result for the hidden layer"""
    return np.dot(W, h) # dot product of weight vector and input vector

## Run the test cases and check the results
for y in x:
    h = hidden(y, W_1)
    print(output(h, W_2))
```

```
1.0
0
-1.0
0
3.0
0
-3.0
1.0
```

If your results agree with the function created above, congratulations! Your first fully connected neural network passed all the tests!

End of exercise.

Notice that even a network to compute a simple function requires 6 weights. You can see that for more complex

functions any practical algorithm must learn a large number of weights. The limitations of Numpy would quickly become evident for large scale problems involving hundreds of millions of weights.

Note: If you are having difficulty following the Numpy code in the above example, you might want to look at [Scott Shell's Numpy Tutorial](#)

Exercise 5-2: You will now construct and test an neural network implementing an [exclusive or function, the XOR](#). The XOR function outputs a 1 if either input is 1 and the other 0, and a 0 otherwise. The truth table for the XOR function is:

$$\begin{matrix} \text{in}_1 & \text{in}_2 & \text{out} \\ 0 & 0 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 1 \\ 1 & 1 & 0 \end{matrix}$$

You can use the `hidden` and `output` functions you created for the previous exercise with new weight tensors. Make sure you try all 4 possible test cases.

```
In [11]: x = [(0,0), (1,0), (0,1), (1,1)]

W_1 = np.array([[1.0, -1.0], [-1.0, 1.0]])

W_2 = np.array([1, 1])

## Run the test cases and check the results
for y in x:
    h = hidden(y, W_1)
    print(output(h, W_2))
```

```
0
1.0
1.0
0
```

If your output agrees with the truth table, congratulations! You have solved the XOR problem using nonlinear activation.

End of exercise.

3.0 Learning in neural networks: Backpropagation

Now that we have a promising representation, we need to determine if it is trainable. The answer is not only yes we can, but that we can do so in a computationally efficient manner, using a clever algorithm known as **backpropagation**.

The backpropagation algorithm was developed independently multiple times. The earliest work on this algorithm was by Kelly (1960) in the context of control theory and Bryson (1961) in the context of dynamic programming. Rumelhart, Hinton and Williams (1984) demonstrated empirically that backpropagation can be used to train neural networks. Their paper marks the beginning of the modern history of neural networks, and set off the first wave of enthusiasm.

The backpropagation algorithm requires several components. First, we need a **loss function** to measure how well our representation matches the function we are trying to learn. Second, we need a way to propagate changes in the representation through the complex network. For this we will use the **chain rule of calculus** to compute **gradients** of the representation. In the general case, this process requires using automatic differentiation methods.

The point of backpropagation is to learn the optimal weight for the neural network. The algorithm proceeds iteratively through a series of small steps. Once we have the gradient of the loss function we can update the tensor of weights.

$$W_{t+1} = W_t + \alpha \nabla_W J(W_t)$$

where

W_t = the tensor of weights or model parameters at step t .

α = step size or learning rate.

$J(W)$ = loss function given the weights.

$\nabla_W J(W)$ = gradient of $J(W)$ with respect to the weights W .

It should be evident that the back propagation algorithm is a form of gradient decent. The weights are updated in

small steps following the gradient of $J(W)$ down hill.

Finally, we need a way evaluate the performance of the model. Without evaluation metrics we have no way to compare the performance of a given model, or compare the performance of several models.

In the next sections, we will address each of loss functions, gradient computation and performance measurement.

3.1 Loss functions

To train a neural network we must have a **loss function**, also known as a **cost function**. In simple terms, the loss function measures the fit of a model to the training data. The lower the loss, the better the fit.

To train deep learning models **cross entropy** is often used as a loss function. This is an information theoretic measure of model fit. We can understand cross entropy as follows.

First define **Shannon entropy** as:

$$\mathbb{H}(X) = E[-\ln_b(P(X))] = - \sum_{i=1}^n P(x_i) \ln_b(P(x_i))$$

Where:

$E[X]$ = the expectation of X .

$I(X)$ = the information content of X .

$P(X)$ = probability of X .

b = base of the logarithm.

This rather abstract formula gives us a way to compute the expected information content of a set of values X . The more likely (higher probability) of X the less informative it is.

To create a loss function from the definition of Shannon entropy we start with the **Kullback-Leibler divergence (KL divergence)** or **relative entropy**. The KL divergence is an information theoretic measure of the difference between two distributions, $P(X)$ and $Q(X)$.

$$\mathbb{D}_{KL}(P \parallel Q) = - \sum_{i=1}^n p(x_i) \ln_b \frac{p(x_i)}{q(x_i)}$$

Ideally, in the case of training a machine learning model we want a distribution $Q(X)$, which is identical to the actual data distribution $P(X)$.

But, you may say, if we could know $P(X)$ why compute $Q(X)$ at all? Fortunately, we do not have to. We can rewrite the KL divergence as:

$$\mathbb{D}_{\text{KL}}(P \parallel Q) = \sum_{i=1}^n p(x_i) \ln_b p(x_i) - \sum_{i=1}^n p(x_i) \ln_b q(x_i)$$

Since $P(X)$ is fixed and we wish to find $Q(X)$ when we train our model, we can minimize the term on the right, which is the **cross entropy** defined as:

$$\mathbb{H}(P, Q) = - \sum_{i=1}^n p(x_i) \ln_b q(x_i)$$

From the formulation of KL divergence above you can see the following.

$$\mathbb{D}_{\text{KL}}(P \parallel Q) = \mathbb{H}(P) + \mathbb{H}(P, Q) \quad \mathbb{D}_{\text{KL}}(P \parallel Q) = \text{Entropy}(P) + \text{Cross Entropy}(P, Q)$$

Thus, we can minimize divergence by minimizing cross entropy. This idea is both intuitive and computationally attractive. The closer the estimated distribution $q(X)$ is to the distribution of the true underlying process $p(X)$, the lower the cross entropy and the lower the KL divergence.

In general we will not know $p(X)$. In fact, if we did, why would we need to solve a training problem? So, we can use the following approximation.

$$\mathbb{H}(P, Q) = - \frac{1}{N} \sum_{i=1}^n \ln_b q(x_i)$$

You may notice, that this approximation, using the average log likelihood, is equivalent to a maximum likelihood estimator (MLE).

Let's look at a specific case of a model with Gaussian likelihood. What is the cross entropy? We can start by thinking about the definition of likelihood.

$$p(\text{data}|\text{model}) = p(\text{data}|\theta) = p(x_i|\hat{\mu}, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{\frac{-(x_i -$$

$$\frac{1}{2\sigma^2}(\hat{\mu} - x_i)^2$$

We take the negative logarithm of this likelihood model.

$$-\log(p(\text{data}|\text{model})) = -\frac{1}{2} \log(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum_{i=1}^n (x_i - \hat{\mu})^2$$

Now, the first term on the right is a constant, as is the denominator of the second term if we assume known variance. Since our goal is to minimize cross entropy, we can eliminate these quantities and be left with just the following.

$$\sum_{i=1}^n (x_i - \hat{\mu})^2$$

This is one issue we need to deal with. Our formulation of cross entropy involves the unknown true distribution of the underlying process $p(X)$. However, since $p(x_i)$ is fixed but unknown we can just write the following.

$$\min_{\mu} \left(-\sum_{i=1}^n \log(p(x_i|\mu)) \right) \propto \min_{\mu} \left(\sum_{i=1}^n (x_i - \hat{\mu})^2 \right)$$

This is just the definition of a Maximum Likelihood Estimator (MLE) for the least squares problem! In fact, since the cross entropy is computed using the negative log likelihood, it will always be minimized by the MLE.

You can see another example of [cross-entropy error function and logistic regression](#).

3.2 Computing Loss functions

The loss function is used to train the model. Therefore the loss function must be computed in an efficient manner.

Given the number of parameters in deep neural nets over-fitting is inevitable. Therefore some regularization is required. We will discuss regularization in greater depth in another lesson. For now, we will just use the following regularized form.

$$\mathbb{H}(P, Q) = J(\theta) = -\frac{1}{N} \sum_{i=1}^N \ln q(x_i|\theta) + \frac{\lambda}{2} \|\theta\|^2 \quad \text{where } -\frac{1}{N} \sum_{i=1}^N \ln q(x_i|\theta) = J_{\text{MLE}}(\theta) \quad \|\theta\|^2 = L^2 \text{ norm regularization term}$$

To minimize $J(\theta)$ in this form θ must be chosen to keep $\|\theta\|^2$ small while minimizing the

negative log likelihood of $q(x_i|\theta)$.

Let's consider how we would compute this form of the loss function. The computational graph shown below illustrates the computational path for the regularized loss function. For simplicity, no bias terms are considered.

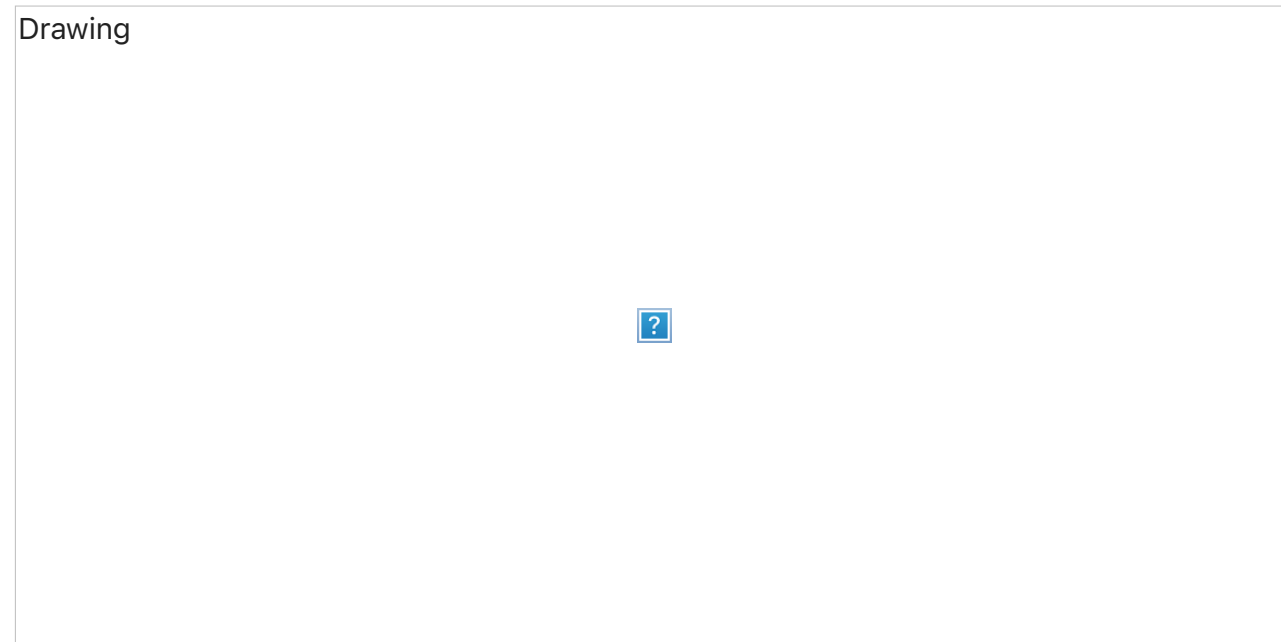


Figure 3.1 Computational graph for computing loss of fully connected neural network of Figure 2.3

3.3 Chain rule of calculus

Key to the back propagation algorithm is the chain rule of calculus; not to be confused with the chain rule of probability. The chain rule allows us to back propagate gradients through an arbitrarily complex graph of functions.

Now, suppose there is a function $y = g(x)$, and another function $z = f(y) = f(g(x))$. How do we compute the derivative of z with respect to x ? Applying the chain rule we get:

$$\frac{dz}{dx} = \frac{dz}{dy} \frac{dy}{dx}$$

Consider $x \in \mathbb{R}^M$ $g(x) \in \mathbb{R}^M$ and $f(y) \in \mathbb{R}$. The chain rule becomes:

$$\frac{\partial z}{\partial x} = \sum_{j \in M} \frac{\partial z}{\partial y_j} \frac{\partial y_j}{\partial x_i}$$

Which we can rewrite as

$$\nabla_x z = \left(\frac{\partial x}{\partial y} \right)^T \nabla_y z$$

Here, $\frac{\partial x}{\partial y}$ is the $n \times m$ **Jacobian matrix** of partial derivatives. The Jacobian is multiplied by the gradient with respect to y , $\nabla_y z$. You can think of the Jacobian as a transformation for a gradient with respect to y to what we really want, the gradient with respect to z .

3.4 Example of finding a gradient.

Let's work out backpropagation for a very simple neural network with a just an input layer and an output layer. This neural network, including the loss function, is shown in Figure 3.2 below. This network has been highly simplified. There are only three layers, input layer, a two unit hidden layer with no bias terms, and a single unit output layer. There are only two weight tensors for this network. Further, the hidden units use rectilinear activation and the output unit uses linear activation. These activation functions have simple partial derivatives.

Drawing



Figure 3.2 Simple single layer neural network with loss function

To analyze this network we will refer to the computational graph shown in Figure 3.1 above.

First, we need to work out the forward propagation relationships. We can compute the outputs of the hidden layer as follows.

$$S_{\{1,2\}} = \sigma_h(W^1 \cdot X_{\{1,2\}}) = \sigma \left(\sum_j W^1_{i,j} x_j \right)$$

In the same way, the result from the output layer can be computed as follows, since the activation function for this layer is linear.

$$S_3 = W^2 \cdot S_{\{1,2\}} = \sum_i W^2_i \sigma \left(\sum_j W^1_{i,j} x_j \right)$$

To perform backpropagation, we need fill out the gradient vector by computing $\frac{\partial J(W)}{\partial W}$ for each weight in the model.

$$\frac{\partial J(W)}{\partial W} = \begin{bmatrix} \frac{\partial J(W)}{\partial W^2_{11}} & \frac{\partial J(W)}{\partial W^2_{12}} & \frac{\partial J(W)}{\partial W^2_{21}} & \frac{\partial J(W)}{\partial W^2_{22}} \end{bmatrix}$$

$$\left\{ \frac{\partial J(W)}{\partial W^1_{12}} \right\}$$

To keep things simple in this example we will just use a non-normalized squared error loss function. This is just the MLE estimator (without normalization) for a Gaussian distribution.

$$J(W) = - \frac{1}{2} \sum_{l=1}^n (y_l - S_{3,l})^2$$

Where:

y_k = the label for the l th case.

$\hat{y}_k = S_{3,k}$ = the output of the network for the l th case.

We want to compute the gradients with respect to the input and output tensors:

$$\frac{\partial J(W)}{\partial W^1}, \frac{\partial J(W)}{\partial W^2}$$

Let's start with the easier case of the partial derivatives with respect to the output tensor. We can apply the chain rule as follows:

$$\frac{\partial J(W)}{\partial W^2_k} = \frac{\partial J(W)}{\partial S_{3,k}} \frac{\partial S_{3,k}}{\partial W^2_k}$$

The first partial derivative of the chain is:

$$\frac{\partial J(W)}{\partial S_{3,k}} = \frac{\partial}{\partial S_{3,k}} \left(-\frac{1}{2} (y_k - S_{3,k})^2 \right) = y_k - S_{3,k}$$

And, the partial derivative of the second partial derivative in the chain, given the linear activation of the output unit:

$$\frac{\partial S_{3,k}}{\partial W^2_k} = \frac{\partial W^2_k S_{j,k}}{\partial W^2_k} = S_{j,l}, \quad j \in \{1, 2\}$$

Multiplying the two components of the chain gives us:

$$\frac{\partial J(W)}{\partial W^2_k} = S_{j,k} (y_k - S_{3,k}), \quad j \in \{1, 2\}$$

The partial derivatives with respect to the input tensor are a bit more complicated. To apply the chain rule we must work backwards from the loss function. This gives the following chain:

$$\frac{\partial J(W)}{\partial W^{1}_{i,j}} = \frac{\partial J(W)}{\partial S_{\{3\}}} \frac{\partial S_{\{3\}}}{\partial S_{\{j\}}} \frac{\partial S_{\{j\}}}{\partial W^{1}_{i,j}}$$

First, we find the right most partial derivative in our chain:

$$\frac{\partial S_j}{\partial W^{1}_{i,j}} = \begin{cases} \frac{\partial W^{1}_{i,j} x_{i,k}}{\partial W^{1}_{i,j}}, & \text{if } S_j > 0 \\ 0, & \text{otherwise} \end{cases}$$

Which given the ReLU activation results in:

$$\frac{\partial S_j}{\partial W^{1}_{i,j}} = \begin{cases} 1, & \text{if } S_j > 0 \\ 0, & \text{otherwise} \end{cases}$$

The middle partial derivative must account for the nonlinearity:

$$\frac{\partial S_{\{3\}}}{\partial S_{\{j\}}} = W^{2}_j$$

We have already computed $\frac{\partial J(W)}{\partial S_{\{3\}}}$. Multiplying all three partial derivatives we find:

$$\frac{\partial J(W)}{\partial W^{1}_{i,j}} = \begin{cases} (y_k - S_{\{3,k\}}) W^{2}_j, & \text{if } S_j > 0 \\ 0, & \text{otherwise} \end{cases}$$

Where $S_{\{3\}}$ and $S_{\{1,2\}}$ are computed using the relationships given above.

A more detailed, but still digestible example of computing gradients for backpropagation can be found in a blog post by [Manfred Zaharaukas](#), among many other places.

4.0 Creating a Model With Keras

You will now create and test a first deep learning classifier model for the MNIST dataset using Keras. The fully connected model has one hidden layer.

4.1 Preparing the Dataset

You have already worked with the MNIST dataset. To load these data and prepare them for the Keras model execute the code in the cell below.

```
In [12]: (train_images, train_labels), (test_images, test_labels) = mnist.load_data()
train_images = train_images.reshape((60000, 28*28)).astype('float32')/255
print(train_images.shape)
test_images = test_images.reshape((10000, 28*28)).astype('float32')/255
print(test_images.shape)
```

```
(60000, 784)
```

```
(10000, 784)
```

```
In [13]: train_images.dtype
```

```
Out[13]: dtype('float32')
```

There is one more preprocessing step. The labels need to be **one hot encoded**. One hot encoding transforms an N level categorical variable into N binary columns. One column represents one category. A 1 or binary true value is encoded in the column of a given category with the other columns coded as 0 or false.

Exercise 5-3: You will now one hot encoded the label vectors of the training and test data. Use the [keras.utils.np_utils.to_categorical](#) function to create the one hot encoded labels. Print the first 10 rows of the training labels. You will need to set options to display all columns.

```
In [14]: train_labels = ku.to_categorical(train_labels)
test_labels = ku.to_categorical(test_labels)
with np.printoptions(threshold=np.inf):
    print(train_labels[:10,:])
```



```
[ [0. 0. 0. 0. 0. 1. 0. 0. 0. 0.]
  [1. 0. 0. 0. 0. 0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.]
  [0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 0. 0. 0. 0. 0. 1.]
  [0. 0. 1. 0. 0. 0. 0. 0. 0. 0.]
  [0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
  [0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
  [0. 1. 0. 0. 0. 0. 0. 0. 0. 0.]
  [0. 0. 0. 0. 1. 0. 0. 0. 0. 0.] ]]
```

Examine the printed one hot encoded labels. Does the number of columns correspond to the number of label categories and why is this expected?

End of exercise.

Answer: Yes, there are 10 columns as is required for encoding 10 unique label categories.

Assignment

4.2 Defining and Executing the Deep Learning Model

It is now time to define the Keras neural network model. This model uses the [Keras Sequential class](#). The model is constructed by **adding dense layers** with the `add` method. Hidden and output layers are specified by creating instances of the [Dense layer class](#). An input data shape must be specified only for the input layer.

► **Exercise 5-4:** You will now specify, [compile and fit](#) the neural network model by the following steps:

1. Define the sequential model by instantiating a model object using `models.Sequential`. Name your model `nn`. Then add layers:

- Add a `keras.layers.Input` layer, with argument `shape=(28*28,)` .
 - Add a `dense input layer` hidden layer with 512 hidden units and rectalinear (`relu`) activation.
 - Add a dense output output layer with 10 units with `activation='softmax'` .
2. `Compile` your model with the following arguments; `optimizer='rmsprop'` , `loss='categorical_crossentropy'` , `metrics=['accuracy']` . We will discuss optimizers in another lesson.
 3. Print a summary of the model with the `summary()` method.
 4. Fit your model, using the `fit()` method, with the training images, training labels, and arguments; `epochs=5` , `batch_size=128` .

In [15]: *## Your code to define the model and display the summary goes here*

```
nn = models.Sequential(name='Exercise-5-4')

nn.add(layers.Input((28*28,)))
nn.add(layers.Dense(512, activation='relu'))
nn.add(layers.Dense(10, activation='softmax'))

nn.summary()
```

Model: "Exercise-5-4"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 512)	401,920
dense_1 (Dense)	(None, 10)	5,130

Total params: 407,050 (1.55 MB)

Trainable params: 407,050 (1.55 MB)

Non-trainable params: 0 (0.00 B)

In [16]: *## Your code to compile and train the model goes here*

```
nn.compile(
```

```
optimizer='rmsprop',
loss='categorical_crossentropy',
metrics=['accuracy']
)

nn.fit(train_images, train_labels, epochs=5, batch_size=128);
```

Epoch 1/5

469/469 ————— 3s 5ms/step – accuracy: 0.8744 – loss: 0.4429

Epoch 2/5

469/469 ————— 2s 4ms/step – accuracy: 0.9659 – loss: 0.1147

Epoch 3/5

469/469 ————— 2s 4ms/step – accuracy: 0.9785 – loss: 0.0747

Epoch 4/5

469/469 ————— 2s 4ms/step – accuracy: 0.9855 – loss: 0.0478

Epoch 5/5

469/469 ————— 2s 3ms/step – accuracy: 0.9886 – loss: 0.0380

```
In [17]: test_loss, test_accuracy = nn.evaluate(test_images, test_labels)
print(f'Test accuracy: {test_accuracy}')
```

313/313 ————— 1s 2ms/step – accuracy: 0.9748 – loss: 0.0780

Test accuracy: 0.979200005531311

```
In [18]: print('The class balance:\n')
print((test_labels.sum(axis=0) / test_labels.sum() * 100).round(2))
```

The class balance:

[9.8 11.35 10.32 10.1 9.82 8.92 9.58 10.28 9.74 10.09]

? Answer these questions:

1. Why is the `categorical_crossentropy` the good choice loss function and `softmax` the correct choice for output activation for this model?
2. Considering the number of training samples, and number of trainable model parameters, where do you think this model might lie along the bias-variance trade-off spectrum?
3. Examine the evolution of the loss function and accuracy. What do these figures tell you about

learning for this model?

End or exercise.

 Answers:

1. The `categorical_crossentropy` is the good choice loss because we're training model to categorize images to one of 10 categories. The categories encoded as a one-hot vector. Softmax assigns each class its probability basing on the model inference, so that all probabilities sum up to 1. That's why Softmax is the most suitable activation for the model output.
2. Considering the number of training samples, and number of trainable model parameters, the model is likely to overfit, which would gravitate it to a high variance side of spectrum. Reason for that is inability to generalize to a new data, that model haven't seen during training. Still basing on the fair balanced test data and test data evaluation above, the model trained surprisingly well, and the overfitting is tolerable. The test accuracy is only a bit less than 2% lower than the train accuracy, while different labels represented almost equally.
3. The evolution of the loss function and accuracy indicates successful model training with training accuracy reaching 99.8%, and validation accuracy just ~2% less than that.

4.3 Performance Metrics

Now that we have the components for training a basic neural network in place we need a way to evaluate its performance. It turns out, there is nothing special about evaluation of neural network models as opposed to other machine learning models. For regression models, one typically use the standard metrics such as root mean square error (RMSE), mean absolute error (MAE). For classification models, one also typically uses the standard metrics including the confusion matrix, accuracy, precision and recall. The [Keras metrics package](#) provides numerous methods for model evaluation.

Execute the code in the cell below to compute and display performance metrics for your model based on the test dataset.

```
In [19]: test_loss, test_accuracy = nn.evaluate(test_images, test_labels)
print("Test accuracy = {0:.3f}    Test loss = {1:.3f}".format(test_accuracy, test_loss))
```

313/313 ————— 1s 2ms/step – accuracy: 0.9748 – loss: 0.0780

Test accuracy = 0.979 Test loss = 0.067

 **Exercise 5-5:** Compare the results of the evaluation with the same metrics achieved during model training. What does the difference tell you about the generalization for this model?

 **Answer:** Both loss and accuracy difference between training and evaluation is reasonably small: \$10\%\$ from 1-st epoch for the loss and \$1.5\%\$ for the accuracy. With still some little space for model quality improvement, this demonstrates reasonably well model generalization (given almost uniform class distribution).

5.0 Diagnosing Neural Network Training History

The Keras model `fit` method creates a [TensorFlow history object using callbacks](#). The information contained in the history object can be very useful in understanding model training; what is working and what is not.

Finding the a good set of hypterparameters for training any deep neural network can be a time consuming taks. **Do not expect your first choice of hypterparameters to be any good!** Further, there is **no cut and paste from one problem to another**. Just because a particular set of hypterparamters resulted in a acceptable learning for one problem using one particular network does not mean that these hyperparameters will be suitable for any other problem or any other neural network.

In the following exercises you will investigate a some commonly required steps to configure hypterparameters to allow a deep neural network toperform reasonable learning. The key to determining if a deep neural network is learning properly to study the evolution of the learning over the epochs. We want the curves of the evaluation metrics, such as loss and validation accuracy, to appear both smooth and with slowly changing slope. Once the model has been trained for a sufficient number of epochs, the metric curves should flatten, having essentially zero slope. The zero slope indicates the learning has ended. If the metric curves appear erratic, you know that the

learning is not proceeding smoothly and is too aggressive. If the curves have abrupt changes in slope this indicates the learning is too aggressive as well. For example, a curve that has a sharp knee and then near zero slope can indicate that learning terminated prematurely.

We will apply the following methods to improve the learning of deep neural network models:

1. **Regularization** is necessary to ensure proper training of deep neural networks. If the model has insufficient regularization, the learning will be erratic with large jumps in the validation metrics, such as error rate and accuracy. On the other hand if too much regularization is applied, learning will be limited by the high bias in the model.
2. **Learning rate** must be set so that learning proceeds slowly and steadily. If the learning is too fast, the learning will terminate prematurely. The, typically local, solution for the weight parameters will be far from optimal. You can identify such cases by a steep curve that flattens out for the performance metrics. In other cases, the aggressive learning will result in erratic curves of the learning metrics. However, if the learning is too slow the neural network will take an excessive number of epochs to reach a steady state where learning has ceased. You can identify such cases by metric curves that never really converge to a zero (flat) slope. However, keep in mind that with complex networks which require learning many parameters, a large number of training epochs may be required.
3. Generally, there is an **interaction between regularization and learning rate**. Changing hyperparameters for regularizers can affect the optimal learning rate and vice versa.

Let's put these concepts into practice.

► **Exercise 5-6:** You will now re-train your model while capturing a history. To retrain a model you must re-compile it first to create a fresh model object. Training an existing model object will continue the training of that object. This property of Keras models can be most useful for improving existing models when new training data becomes available. Do the following:

1. Compile the existing model using the same arguments as before.
2. Fit the model as before, but for 20 epochs and with an additional argument; `validation_data=(test_images, test_labels)`. Assign the results to a variable, `history_nn`.

3. Execute the code in the next two cells to display charts of training and test loss and accuracy.

```
In [20]: ## Your code to compile and train the model goes here
nn = models.Sequential(name='Exercise-5-6')

nn.add(layers.Input((28*28,)))
nn.add(layers.Dense(512, activation='relu'))
nn.add(layers.Dense(10, activation='softmax'))

nn.compile(
    optimizer='rmsprop',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_nn = nn.fit(
    train_images,
    train_labels,
    validation_data=(test_images, test_labels),
    epochs=20,
    batch_size=128
);
```

Epoch 1/20

469/469 ————— 2s 4ms/step - accuracy: 0.8714 - loss: 0.4442 - val_accuracy: 0.9632 - val_loss: 0.1233

Epoch 2/20

469/469 ————— 2s 4ms/step - accuracy: 0.9663 - loss: 0.1153 - val_accuracy: 0.9735 - val_loss: 0.0917

Epoch 3/20

469/469 ————— 2s 4ms/step - accuracy: 0.9785 - loss: 0.0725 - val_accuracy: 0.9766 - val_loss: 0.0773

Epoch 4/20

469/469 ————— 2s 4ms/step - accuracy: 0.9844 - loss: 0.0538 - val_accuracy: 0.9761 - val_loss: 0.0761

Epoch 5/20

469/469 ————— 2s 4ms/step - accuracy: 0.9886 - loss: 0.0390 - val_accuracy: 0.9741 - val_loss: 0.0802

```

Epoch 6/20
469/469  2s 4ms/step - accuracy: 0.9923 - loss: 0.0273 - val_accuracy: 0.9801 -
val_loss: 0.0656
Epoch 7/20
469/469  2s 4ms/step - accuracy: 0.9942 - loss: 0.0209 - val_accuracy: 0.9798 -
val_loss: 0.0671
Epoch 8/20
469/469  2s 4ms/step - accuracy: 0.9961 - loss: 0.0154 - val_accuracy: 0.9798 -
val_loss: 0.0722
Epoch 9/20
469/469  2s 4ms/step - accuracy: 0.9972 - loss: 0.0115 - val_accuracy: 0.9832 -
val_loss: 0.0567
Epoch 10/20
469/469  2s 4ms/step - accuracy: 0.9979 - loss: 0.0090 - val_accuracy: 0.9833 -
val_loss: 0.0625
Epoch 11/20
469/469  2s 4ms/step - accuracy: 0.9986 - loss: 0.0064 - val_accuracy: 0.9818 -
val_loss: 0.0650
Epoch 12/20
469/469  2s 4ms/step - accuracy: 0.9989 - loss: 0.0052 - val_accuracy: 0.9837 -
val_loss: 0.0611
Epoch 13/20
469/469  2s 4ms/step - accuracy: 0.9995 - loss: 0.0032 - val_accuracy: 0.9823 -
val_loss: 0.0637
Epoch 14/20
469/469  2s 4ms/step - accuracy: 0.9996 - loss: 0.0024 - val_accuracy: 0.9834 -
val_loss: 0.0629
Epoch 15/20
469/469  2s 4ms/step - accuracy: 0.9997 - loss: 0.0019 - val_accuracy: 0.9830 -
val_loss: 0.0650
Epoch 16/20
469/469  2s 4ms/step - accuracy: 0.9999 - loss: 0.0012 - val_accuracy: 0.9831 -
val_loss: 0.0663
Epoch 17/20
469/469  2s 4ms/step - accuracy: 1.0000 - loss: 8.2178e-04 - val_accuracy: 0.98
33 - val_loss: 0.0652
Epoch 18/20
469/469  2s 4ms/step - accuracy: 1.0000 - loss: 4.8005e-04 - val_accuracy: 0.98
34 - val_loss: 0.0667

```


Epoch 19/20

469/469 ————— 2s 4ms/step - accuracy: 1.0000 - loss: 3.8725e-04 - val_accuracy: 0.98
40 - val_loss: 0.0657

Epoch 20/20

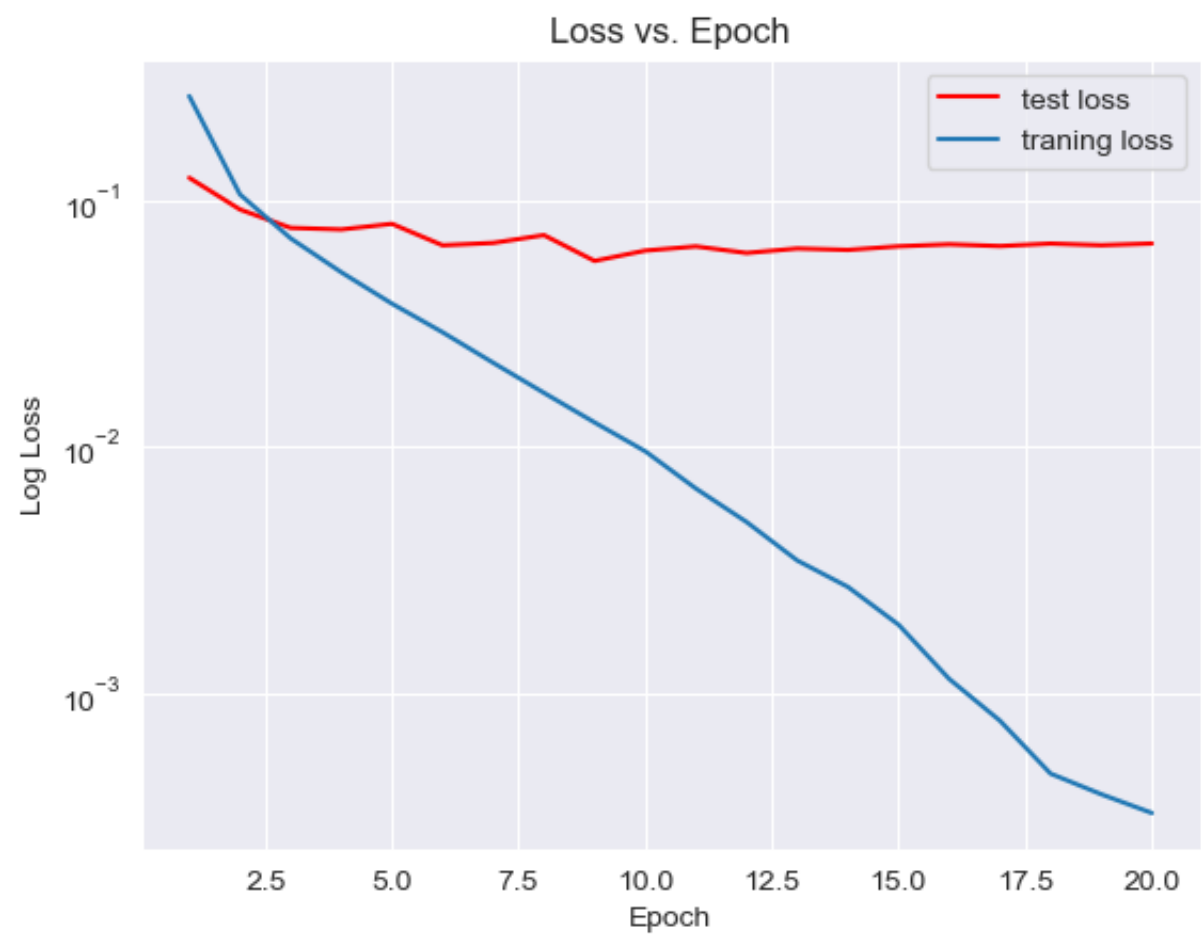
469/469 ————— 2s 4ms/step - accuracy: 1.0000 - loss: 3.1238e-04 - val_accuracy: 0.98
39 - val_loss: 0.0668

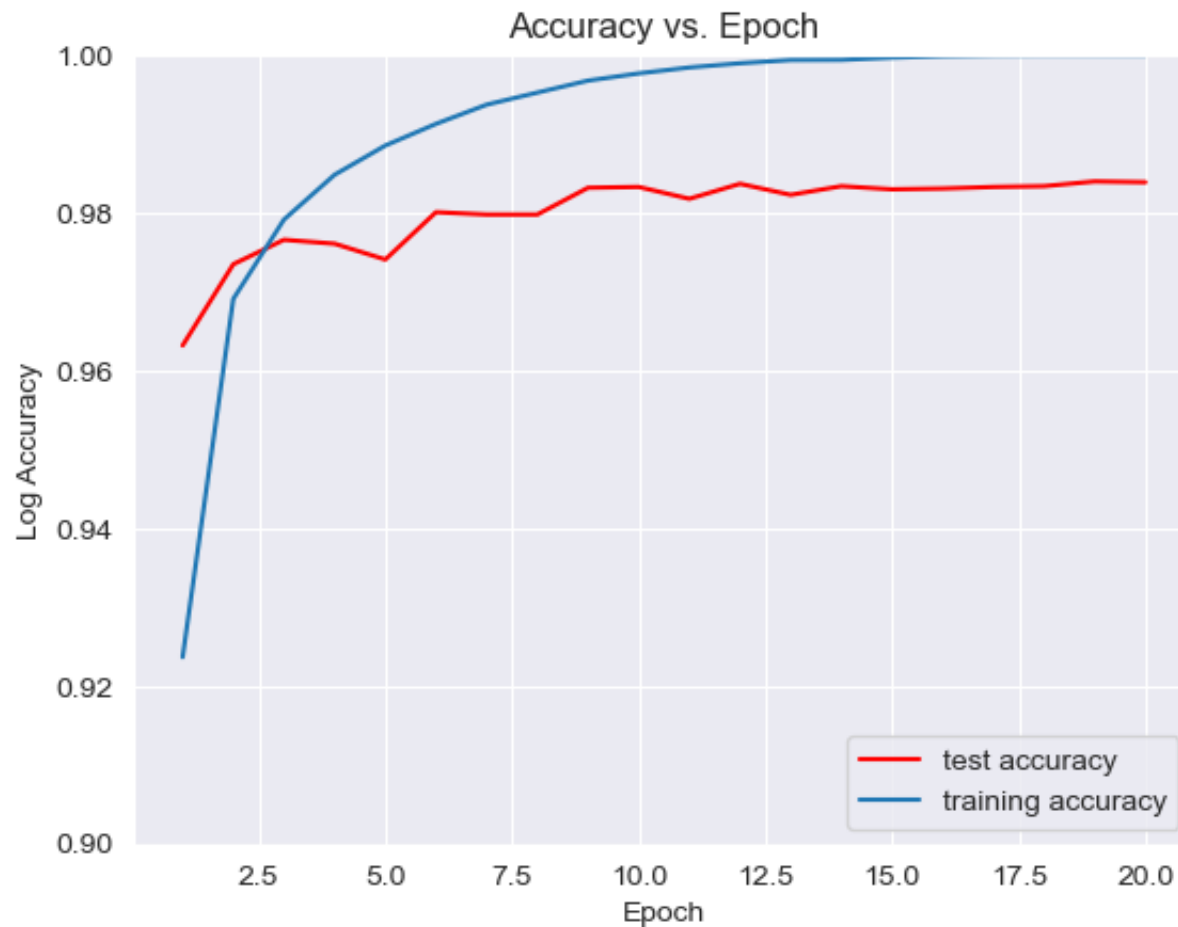
Next, execute the code in the cell below to plot the training and validation loss and accuracy histories.

```
In [21]: def plot_loss(history):
    train_loss = history.history['loss']
    test_loss = history.history['val_loss']
    x = list(range(1, len(test_loss) + 1))
    plt.plot(x, test_loss, color = 'red', label = 'test loss')
    plt.plot(x, train_loss, label = 'training loss')
    plt.yscale('log')
    plt.xlabel('Epoch')
    plt.ylabel('Log Loss')
    plt.title('Loss vs. Epoch')
    plt.legend()
    plt.show()

def plot_accuracy(history):
    train_acc = history.history['accuracy']
    test_acc = history.history['val_accuracy']
    x = list(range(1, len(test_acc) + 1))
    plt.plot(x, test_acc, color = 'red', label = 'test accuracy')
    plt.plot(x, train_acc, label = 'training accuracy')
    plt.ylim(0.9, 1.0)
    plt.xlabel('Epoch')
    plt.ylabel('Log Accuracy')
    plt.title('Accuracy vs. Epoch')
    plt.legend(loc='lower right')
    plt.show()

plot_loss(history_nn)
plot_accuracy(history_nn)
```





✅ I've updated the `plot_accuracy(...)` method, removing Y-axis log scale and fixing its range from 0.9 to 1, to make comparing different model runs easier. Since before that different exercise accuracy history plots had different scale and different Y-labels even.

❓ Examine the trajectory of the training and test loss and accuracy. What does the erratic and divergent trajectories of the loss and accuracy tell you about the learning and generalization of the model?

End of exercise.

✅ **Answer:** The erratic test accuracy curve, stabilizing around 10-th epoch shows that at the later epochs model become more stable on the validation data, which wasn't the case at first 10 epochs. The fact that Accuracy don't improve during epochs 5-20 indicates overfitting. It makes sense to apply regularization techniques, like adjustable learning rate, L1 and L2 regularization, drop-out and batch optimisation.

6.0 Adding Regularization

You have seen some of the effects of over-fitting of the neural network model. **Regularization** methods are widely used in machine learning to prevent over-fitting. Conceptually, you can think of regularization as moving the model toward lower variance and higher bias to improve generalization. There are several widely used regularization methods for deep learning models. These methods are generally used in combinations.

- **L1 and L2 regularization** which we examined in some detail in the previous lesson.
- **Weight decay** exponentially decays the learned weights toward 0. This process constrains the learning and prevents over-fitting. Any unusually large weights will decay back toward zero and therefore more reasonable values.
- **Batch Normalization** adjusts the output of a layer to ensure it is zero mean and unit variance. This process ensures that the results of the activations of a layer are not excessive that therefore result in over-fitting.
- **Dropout regularization** randomly drops connections (weights) between layers as the learning proceeds. The resulting model is then an ensemble of the weights learned from each of the randomly created models. This ensemble improves regularization and generalization of the model.

With so many options a search of possible regularization methods and hyperparameters is required. This search requires running a particular neural network architecture multiple times as hyperparameter combinations are tested. There are a number of approaches one can take to this search.

- **Grid search** wherein a regular grid of hyperparameter choices are tried. Grid search has the advantage of being comprehensive, but at a high computational cost.
- **Random sampling** wherein a grid of possible hyperparameter choices is randomly sampled and tested. This

method can be significantly faster than a grid search, and is often nearly as effective.

- **Huristic search** is often performed manually, selecting hyperparamter combinations based on the results of previous combinations. We will employ huristic search in these exercises because of the simplicity.

In the case used for the following exercise, the following hyperparameter combinations where tried. Keep in mind, the exact validation metrics achieved will vary from one run to the next as a result of random sampling effects.

Validatiion Accuracy	L2 regularization	Learning rate	Weight decay	Batch normalization
.8979	0.05	0.00005	0.0001	None
.8754	0.5	0.000005	None	None
.8763	0.05	0.000005	0.0001	None
Erratic learning	0.005	0.0005	0.0001	None
.9462	0.005	0.0005	0.01	None

Notice that the hypterparameters control several aspects of the model training.

1. The L2 affects the weights learned for the [Dense](#) layer. Notice that regularizers can be applied to other weights, such as the bias. We will ignore these alternatives for now.
2. The momentum hypterparameter is the primary control for the behavior of the [BatchNormalization](#) layer.
3. The [RMSprop](#) optimizer has quite a number of hypterparameters. The learning rate and weight decay are the most important in general. Selecting an optimal value of momentum can be helpful in some cases. Other hypterparameters are helpful for cases where the optimizer fails to converge. It should be noted that it is not unusual for a particular choice of optimizer to not work well for a specific problem and neural network. Another optimizer can then be tried. Keras, like most deep learning frameworks, has a [long list of optimizers](#) you can choose from.

Important note: In the following examples we limit the search for optimal hyperparamters in several ways.

1. We will only search a limited number of architectural alternatives. In this case, we will test models

with and without batch normalization, a form of regularization. For a real-world problem, a more extensive search for optimal model architecture is often required. For example, we could try greater model depth, in the form of more fully connected layers, or we could experiment with adding dropout regularization layers.

2. We will only search a limited hyperparameter space. A full search would take considerable computing time, but could be worth the improvement if we were optimizing a model for real-world use.
3. We are limiting training time to just 80 epochs. For complex models it is typical to train for hundreds or even thousands of epochs.
4. Do not expect to get exactly the results shown for any of these models. As a result of random initial weights and the stochastic min-batch sampling results will vary from run to run.

► **Exercise 5-7:** You will now add L2 combined with weight decay regularization to the hidden layers of your model. Keras uses the [layer weight regularizer class](#) to add L1 and L2 weight constraints to the layers. Here, we will only use the L2 regularizer. At the same time we must change the learning rate. Do the following:

1. Starting with the model specification you have been using for exercise 5-6, add the following argument to the 512 unit hidden layer specification;
`kernel_regularizer=regularizers.l2(0.005)`.
2. Compile the model with the 'optimizer=RMSprop(learning_rate=0.0005, weight_decay=0.01)' argument. This code sets a learning rate and a weight decay parameter.
3. Fit the model for 80 epochs, saving the history object. This will take some time!
4. Plot the training and test loss and accuracy.

Note on compatibility. More recent versions of TensorFlow use a `weight_decay` replacing the older `weight` argument. If you find an error of an unknown argument try changing to the other possibility.

```
In [22]: ## Your code to define the model and display the summary goes here
```

```

nn = models.Sequential(name='Exercise-5-7')

nn.add(layers.Input((28*28,)))
nn.add(layers.Dense(512, activation='relu', kernel_regularizer=regularizers.l2(0.005)))
nn.add(layers.Dense(10, activation='softmax'))

nn.compile(
    optimizer=RMSprop(learning_rate=0.0005, weight_decay=0.01),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_nn = nn.fit(
    train_images,
    train_labels,
    validation_data=(test_images, test_labels),
    epochs=80,
    batch_size=128
);

test_loss, test_accuracy = nn.evaluate(test_images, test_labels)
print("Test accuracy = {0:.3f}    Test loss = {1:.3f}".format(test_accuracy, test_loss))

```

Epoch 1/80

469/469 ————— 3s 5ms/step - accuracy: 0.8468 - loss: 1.9152 - val_accuracy: 0.9323 - val_loss: 0.4166

Epoch 2/80

469/469 ————— 3s 5ms/step - accuracy: 0.9311 - loss: 0.3941 - val_accuracy: 0.9414 - val_loss: 0.3197

Epoch 3/80

469/469 ————— 2s 5ms/step - accuracy: 0.9456 - loss: 0.3057 - val_accuracy: 0.9470 - val_loss: 0.2820

Epoch 4/80

469/469 ————— 2s 5ms/step - accuracy: 0.9520 - loss: 0.2728 - val_accuracy: 0.9598 - val_loss: 0.2375

Epoch 5/80

469/469 ————— 3s 5ms/step - accuracy: 0.9571 - loss: 0.2470 - val_accuracy: 0.9587 - val_loss: 0.2344

Epoch 6/80

469/469 ————— 3s 6ms/step - accuracy: 0.9601 - loss: 0.2335 - val_accuracy: 0.9537 -

```

val_loss: 0.2364
Epoch 7/80
469/469 ————— 2s 5ms/step - accuracy: 0.9616 - loss: 0.2218 - val_accuracy: 0.9572 -
val_loss: 0.2270
Epoch 8/80
469/469 ————— 3s 5ms/step - accuracy: 0.9629 - loss: 0.2154 - val_accuracy: 0.9639 -
val_loss: 0.2084
Epoch 9/80
469/469 ————— 2s 5ms/step - accuracy: 0.9644 - loss: 0.2102 - val_accuracy: 0.9650 -
val_loss: 0.2041
Epoch 10/80
469/469 ————— 2s 5ms/step - accuracy: 0.9651 - loss: 0.2055 - val_accuracy: 0.9570 -
val_loss: 0.2222
Epoch 11/80
469/469 ————— 2s 5ms/step - accuracy: 0.9663 - loss: 0.1955 - val_accuracy: 0.9690 -
val_loss: 0.1894
Epoch 12/80
469/469 ————— 3s 5ms/step - accuracy: 0.9673 - loss: 0.1919 - val_accuracy: 0.9624 -
val_loss: 0.2007
Epoch 13/80
469/469 ————— 3s 5ms/step - accuracy: 0.9694 - loss: 0.1866 - val_accuracy: 0.9624 -
val_loss: 0.2093
Epoch 14/80
469/469 ————— 3s 5ms/step - accuracy: 0.9686 - loss: 0.1842 - val_accuracy: 0.9652 -
val_loss: 0.1916
Epoch 15/80
469/469 ————— 2s 5ms/step - accuracy: 0.9706 - loss: 0.1803 - val_accuracy: 0.9699 -
val_loss: 0.1689
Epoch 16/80
469/469 ————— 2s 5ms/step - accuracy: 0.9689 - loss: 0.1786 - val_accuracy: 0.9627 -
val_loss: 0.1939
Epoch 17/80
469/469 ————— 3s 5ms/step - accuracy: 0.9704 - loss: 0.1738 - val_accuracy: 0.9678 -
val_loss: 0.1814
Epoch 18/80
469/469 ————— 3s 6ms/step - accuracy: 0.9697 - loss: 0.1759 - val_accuracy: 0.9555 -
val_loss: 0.2187
Epoch 19/80
469/469 ————— 3s 5ms/step - accuracy: 0.9718 - loss: 0.1702 - val_accuracy: 0.9618 -

```



```

val_loss: 0.2093
Epoch 20/80
469/469 ————— 3s 5ms/step - accuracy: 0.9721 - loss: 0.1684 - val_accuracy: 0.9695 -
val_loss: 0.1735
Epoch 21/80
469/469 ————— 3s 6ms/step - accuracy: 0.9719 - loss: 0.1656 - val_accuracy: 0.9727 -
val_loss: 0.1639
Epoch 22/80
469/469 ————— 3s 6ms/step - accuracy: 0.9715 - loss: 0.1680 - val_accuracy: 0.9689 -
val_loss: 0.1712
Epoch 23/80
469/469 ————— 2s 5ms/step - accuracy: 0.9738 - loss: 0.1593 - val_accuracy: 0.9730 -
val_loss: 0.1602
Epoch 24/80
469/469 ————— 2s 5ms/step - accuracy: 0.9727 - loss: 0.1632 - val_accuracy: 0.9697 -
val_loss: 0.1715
Epoch 25/80
469/469 ————— 2s 5ms/step - accuracy: 0.9733 - loss: 0.1597 - val_accuracy: 0.9677 -
val_loss: 0.1788
Epoch 26/80
469/469 ————— 3s 5ms/step - accuracy: 0.9733 - loss: 0.1588 - val_accuracy: 0.9704 -
val_loss: 0.1631
Epoch 27/80
469/469 ————— 3s 5ms/step - accuracy: 0.9736 - loss: 0.1567 - val_accuracy: 0.9714 -
val_loss: 0.1574
Epoch 28/80
469/469 ————— 2s 5ms/step - accuracy: 0.9736 - loss: 0.1585 - val_accuracy: 0.9635 -
val_loss: 0.1821
Epoch 29/80
469/469 ————— 3s 6ms/step - accuracy: 0.9741 - loss: 0.1553 - val_accuracy: 0.9656 -
val_loss: 0.1726
Epoch 30/80
469/469 ————— 3s 6ms/step - accuracy: 0.9747 - loss: 0.1536 - val_accuracy: 0.9739 -
val_loss: 0.1534
Epoch 31/80
469/469 ————— 3s 5ms/step - accuracy: 0.9745 - loss: 0.1500 - val_accuracy: 0.9743 -
val_loss: 0.1499
Epoch 32/80
469/469 ————— 3s 5ms/step - accuracy: 0.9751 - loss: 0.1482 - val_accuracy: 0.9712 -

```

```

val_loss: 0.1622
Epoch 33/80
469/469 ————— 3s 5ms/step - accuracy: 0.9768 - loss: 0.1450 - val_accuracy: 0.9747 -
val_loss: 0.1501
Epoch 34/80
469/469 ————— 3s 5ms/step - accuracy: 0.9761 - loss: 0.1487 - val_accuracy: 0.9746 -
val_loss: 0.1497
Epoch 35/80
469/469 ————— 3s 5ms/step - accuracy: 0.9761 - loss: 0.1459 - val_accuracy: 0.9731 -
val_loss: 0.1540
Epoch 36/80
469/469 ————— 3s 5ms/step - accuracy: 0.9766 - loss: 0.1453 - val_accuracy: 0.9753 -
val_loss: 0.1415
Epoch 37/80
469/469 ————— 3s 5ms/step - accuracy: 0.9744 - loss: 0.1487 - val_accuracy: 0.9729 -
val_loss: 0.1514
Epoch 38/80
469/469 ————— 2s 5ms/step - accuracy: 0.9755 - loss: 0.1462 - val_accuracy: 0.9739 -
val_loss: 0.1518
Epoch 39/80
469/469 ————— 3s 5ms/step - accuracy: 0.9763 - loss: 0.1409 - val_accuracy: 0.9697 -
val_loss: 0.1631
Epoch 40/80
469/469 ————— 3s 5ms/step - accuracy: 0.9765 - loss: 0.1435 - val_accuracy: 0.9738 -
val_loss: 0.1505
Epoch 41/80
469/469 ————— 3s 5ms/step - accuracy: 0.9756 - loss: 0.1443 - val_accuracy: 0.9751 -
val_loss: 0.1482
Epoch 42/80
469/469 ————— 3s 5ms/step - accuracy: 0.9782 - loss: 0.1399 - val_accuracy: 0.9710 -
val_loss: 0.1555
Epoch 43/80
469/469 ————— 3s 6ms/step - accuracy: 0.9773 - loss: 0.1399 - val_accuracy: 0.9758 -
val_loss: 0.1436
Epoch 44/80
469/469 ————— 3s 6ms/step - accuracy: 0.9773 - loss: 0.1396 - val_accuracy: 0.9722 -
val_loss: 0.1494
Epoch 45/80
469/469 ————— 3s 5ms/step - accuracy: 0.9773 - loss: 0.1392 - val_accuracy: 0.9702 -

```

```

val_loss: 0.1572
Epoch 46/80
469/469 ————— 3s 5ms/step - accuracy: 0.9770 - loss: 0.1370 - val_accuracy: 0.9749 -
val_loss: 0.1428
Epoch 47/80
469/469 ————— 3s 5ms/step - accuracy: 0.9770 - loss: 0.1389 - val_accuracy: 0.9772 -
val_loss: 0.1375
Epoch 48/80
469/469 ————— 3s 5ms/step - accuracy: 0.9786 - loss: 0.1346 - val_accuracy: 0.9752 -
val_loss: 0.1426
Epoch 49/80
469/469 ————— 3s 5ms/step - accuracy: 0.9777 - loss: 0.1346 - val_accuracy: 0.9648 -
val_loss: 0.1763
Epoch 50/80
469/469 ————— 3s 5ms/step - accuracy: 0.9790 - loss: 0.1340 - val_accuracy: 0.9763 -
val_loss: 0.1387
Epoch 51/80
469/469 ————— 3s 5ms/step - accuracy: 0.9779 - loss: 0.1346 - val_accuracy: 0.9655 -
val_loss: 0.1697
Epoch 52/80
469/469 ————— 3s 5ms/step - accuracy: 0.9778 - loss: 0.1342 - val_accuracy: 0.9760 -
val_loss: 0.1423
Epoch 53/80
469/469 ————— 3s 5ms/step - accuracy: 0.9775 - loss: 0.1347 - val_accuracy: 0.9761 -
val_loss: 0.1435
Epoch 54/80
469/469 ————— 3s 5ms/step - accuracy: 0.9789 - loss: 0.1318 - val_accuracy: 0.9740 -
val_loss: 0.1475
Epoch 55/80
469/469 ————— 2s 5ms/step - accuracy: 0.9783 - loss: 0.1303 - val_accuracy: 0.9778 -
val_loss: 0.1334
Epoch 56/80
469/469 ————— 3s 5ms/step - accuracy: 0.9785 - loss: 0.1309 - val_accuracy: 0.9699 -
val_loss: 0.1541
Epoch 57/80
469/469 ————— 3s 5ms/step - accuracy: 0.9769 - loss: 0.1341 - val_accuracy: 0.9566 -
val_loss: 0.1923
Epoch 58/80
469/469 ————— 3s 5ms/step - accuracy: 0.9783 - loss: 0.1312 - val_accuracy: 0.9660 -

```

```

val_loss: 0.1624
Epoch 59/80
469/469  3s 5ms/step - accuracy: 0.9799 - loss: 0.1297 - val_accuracy: 0.9761 -
val_loss: 0.1359
Epoch 60/80
469/469  3s 5ms/step - accuracy: 0.9787 - loss: 0.1298 - val_accuracy: 0.9757 -
val_loss: 0.1414
Epoch 61/80
469/469  3s 5ms/step - accuracy: 0.9782 - loss: 0.1301 - val_accuracy: 0.9753 -
val_loss: 0.1388
Epoch 62/80
469/469  3s 5ms/step - accuracy: 0.9788 - loss: 0.1294 - val_accuracy: 0.9682 -
val_loss: 0.1587
Epoch 63/80
469/469  3s 5ms/step - accuracy: 0.9794 - loss: 0.1293 - val_accuracy: 0.9762 -
val_loss: 0.1367
Epoch 64/80
469/469  3s 5ms/step - accuracy: 0.9788 - loss: 0.1280 - val_accuracy: 0.9686 -
val_loss: 0.1582
Epoch 65/80
469/469  3s 5ms/step - accuracy: 0.9792 - loss: 0.1254 - val_accuracy: 0.9754 -
val_loss: 0.1371
Epoch 66/80
469/469  3s 5ms/step - accuracy: 0.9791 - loss: 0.1283 - val_accuracy: 0.9746 -
val_loss: 0.1403
Epoch 67/80
469/469  3s 5ms/step - accuracy: 0.9792 - loss: 0.1263 - val_accuracy: 0.9720 -
val_loss: 0.1498
Epoch 68/80
469/469  3s 5ms/step - accuracy: 0.9792 - loss: 0.1276 - val_accuracy: 0.9787 -
val_loss: 0.1234
Epoch 69/80
469/469  3s 5ms/step - accuracy: 0.9804 - loss: 0.1268 - val_accuracy: 0.9743 -
val_loss: 0.1365
Epoch 70/80
469/469  5s 5ms/step - accuracy: 0.9796 - loss: 0.1266 - val_accuracy: 0.9715 -
val_loss: 0.1505
Epoch 71/80
469/469  3s 5ms/step - accuracy: 0.9800 - loss: 0.1244 - val_accuracy: 0.9716 -

```

```

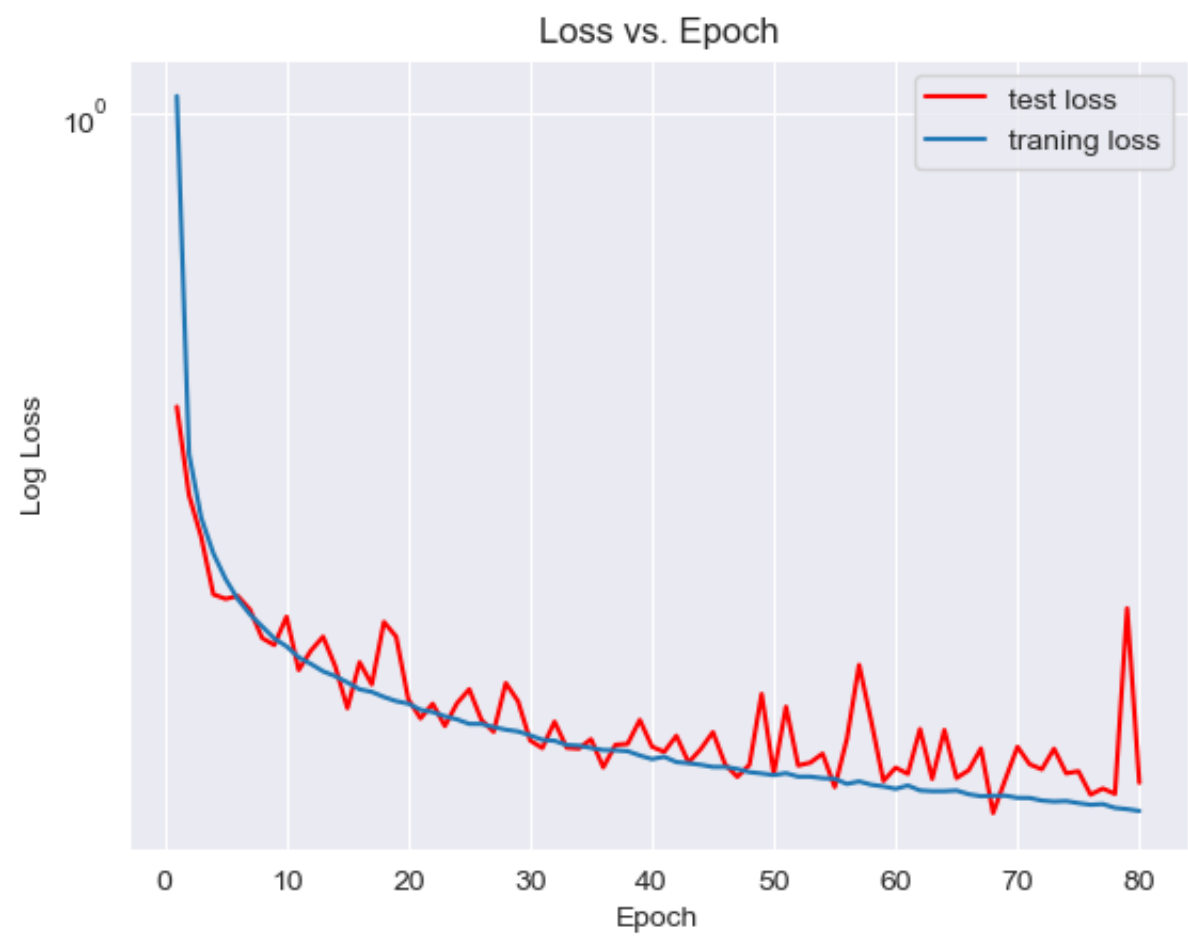
val_loss: 0.1428
Epoch 72/80
469/469 ————— 5s 5ms/step - accuracy: 0.9804 - loss: 0.1218 - val_accuracy: 0.9753 -
val_loss: 0.1407
Epoch 73/80
469/469 ————— 2s 5ms/step - accuracy: 0.9790 - loss: 0.1255 - val_accuracy: 0.9708 -
val_loss: 0.1496
Epoch 74/80
469/469 ————— 3s 6ms/step - accuracy: 0.9798 - loss: 0.1244 - val_accuracy: 0.9754 -
val_loss: 0.1390
Epoch 75/80
469/469 ————— 3s 6ms/step - accuracy: 0.9807 - loss: 0.1215 - val_accuracy: 0.9751 -
val_loss: 0.1399
Epoch 76/80
469/469 ————— 3s 5ms/step - accuracy: 0.9799 - loss: 0.1225 - val_accuracy: 0.9765 -
val_loss: 0.1304
Epoch 77/80
469/469 ————— 3s 5ms/step - accuracy: 0.9803 - loss: 0.1213 - val_accuracy: 0.9764 -
val_loss: 0.1328
Epoch 78/80
469/469 ————— 3s 5ms/step - accuracy: 0.9802 - loss: 0.1220 - val_accuracy: 0.9775 -
val_loss: 0.1307
Epoch 79/80
469/469 ————— 3s 5ms/step - accuracy: 0.9807 - loss: 0.1200 - val_accuracy: 0.9439 -
val_loss: 0.2278
Epoch 80/80
469/469 ————— 3s 5ms/step - accuracy: 0.9808 - loss: 0.1211 - val_accuracy: 0.9745 -
val_loss: 0.1352
313/313 ————— 1s 2ms/step - accuracy: 0.9676 - loss: 0.1515
Test accuracy = 0.975   Test loss = 0.135

```

```

In [23]: plot_loss(history_nn)
         plot_accuracy(history_nn)

```





? Compare the charts of loss and accuracy for the regularized model with those from training the unregularized model. The shape of the training and validation metric curves is greatly improved. Now, answer these questions:

1. Do the charts and numeric metrics for the regularized model show an improvement in the generalization of the model compared to the unregularized model, and why?
2. Notice that the test and training loss of the regularized model continue to improve. Do you think that training for additional epochs will be beneficial?
3. Do the validation metrics show any sign of randomness and what does this tell you about the

model architecture and the choice of hyperparameters?

End of exercise.

 Answers:

1. Unregularized model performance is better so far. Given the high variance on the validation data, the reason is overfitting, which is a consequence of non-optimal model hyper-params.
2. I'm not sure that addition epochs will do significantly better. Even though train performance goes up, the validation accuracy seems to fluctuate around the horizontal asymptotic line.
3. The validation metrics definitely show randomness, which is a clear indication that we can pick different hyperparameters for a better fit.

As a next step to improve the model we will add a batch normalization layer following the fully connected hidden layer. As a result of this change in architecture, we must perform another hyperparameter search. In this case, another heuristic search is performed with a fixed momentum hyperparameter for the batch normalization. The results of the search are shown in the table.

Validation Accuracy	L2 regularization	Learning rate	Weight decay	Batch normalization
.9752	0.2	0.00005	None	Momentum=0.998
Erratic learning	0.2	0.0005	0.001	Momentum=0.998
Slow learning	0.02	0.00005	0.01	Momentum=0.998
Slow learning	0.02	0.00005	0.01	Momentum=0.998
.9789	0.2	0.0005	0.01	Momentum=0.998

 **Exercise 5-8:** The path of the test loss and particularly the test accuracy of the foregoing model is rather erratic. Such behavior often indicates that the gradient is difficult to estimate and therefore the optimizer exhibits poor convergence. Batch normalization is known to smooth the loss function which improves the ability to consistently estimate gradient. You will now add a **BatchNormalization layer** to

the model, along with L2 regularization. Do the following:

1. Start with the model specification you used for Exercise 5-7, change the model name to `nnbr`.
2. For the dense hidden layer set the L2 regularization hyperparameter to 0.2.
3. After the 512 unit hidden dense layer add a `BatchNormalization` layer, using the argument `momentum=0.998`.
4. Compile the model with the optimizer=`RMSprop(learning_rate=0.0005, weight_decay=0.01)`, argument.
5. Fit the model for 80 epochs, saving the history object. This will take some time!
6. Plot the training and test loss and accuracy.

In [24]: *## Your code to define the model and display the summary goes here*

```
nnbr = models.Sequential(name='Exercise-5-8')

nnbr.add(layers.Input((28*28,)))
nnbr.add(layers.Dense(512, kernel_regularizer=regularizers.l2(0.2)))
nnbr.add(layers.BatchNormalization(momentum=0.998))
nnbr.add(layers.Activation('relu'))
nnbr.add(layers.Dense(10, activation='softmax'))

nnbr.compile(
    optimizer=RMSprop(learning_rate=0.0005, weight_decay=0.01),
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_nnbr = nnbr.fit(
    train_images,
    train_labels,
    validation_data=(test_images, test_labels),
    epochs=80,
    batch_size=128
);

test_loss, test_accuracy = nnbr.evaluate(test_images, test_labels)
```

```
print("Test accuracy = {0:.3f}   Test loss = {1:.3f}".format(test_accuracy, test_loss))
```

Epoch 1/80

469/469 ————— 4s 7ms/step - accuracy: 0.8410 - loss: 27.7633 - val_accuracy: 0.8355 - val_loss: 1.8518

Epoch 2/80

469/469 ————— 3s 7ms/step - accuracy: 0.9262 - loss: 0.4563 - val_accuracy: 0.8794 - val_loss: 1.5325

Epoch 3/80

469/469 ————— 3s 7ms/step - accuracy: 0.9321 - loss: 0.4379 - val_accuracy: 0.8984 - val_loss: 1.0930

Epoch 4/80

469/469 ————— 3s 7ms/step - accuracy: 0.9294 - loss: 0.4453 - val_accuracy: 0.9201 - val_loss: 0.7379

Epoch 5/80

469/469 ————— 3s 7ms/step - accuracy: 0.9322 - loss: 0.4335 - val_accuracy: 0.8604 - val_loss: 0.6683

Epoch 6/80

469/469 ————— 3s 7ms/step - accuracy: 0.9372 - loss: 0.4225 - val_accuracy: 0.8931 - val_loss: 0.5672

Epoch 7/80

469/469 ————— 3s 7ms/step - accuracy: 0.9381 - loss: 0.4172 - val_accuracy: 0.8960 - val_loss: 0.5345

Epoch 8/80

469/469 ————— 3s 7ms/step - accuracy: 0.9386 - loss: 0.4154 - val_accuracy: 0.9158 - val_loss: 0.4729

Epoch 9/80

469/469 ————— 3s 7ms/step - accuracy: 0.9410 - loss: 0.3957 - val_accuracy: 0.9337 - val_loss: 0.4275

Epoch 10/80

469/469 ————— 3s 7ms/step - accuracy: 0.9402 - loss: 0.3935 - val_accuracy: 0.9238 - val_loss: 0.4559

Epoch 11/80

469/469 ————— 3s 7ms/step - accuracy: 0.9427 - loss: 0.3770 - val_accuracy: 0.8917 - val_loss: 0.5660

Epoch 12/80

469/469 ————— 3s 7ms/step - accuracy: 0.9454 - loss: 0.3679 - val_accuracy: 0.9181 - val_loss: 0.4476

Epoch 13/80

469/469 ————— 3s 7ms/step - accuracy: 0.9460 - loss: 0.3628 - val_accuracy: 0.8355 -

```
val_loss: 0.7303
Epoch 14/80
469/469  3s 7ms/step - accuracy: 0.9460 - loss: 0.3582 - val_accuracy: 0.8885 -
val_loss: 0.5368
Epoch 15/80
469/469  4s 8ms/step - accuracy: 0.9471 - loss: 0.3464 - val_accuracy: 0.8871 -
val_loss: 0.5237
Epoch 16/80
469/469  4s 8ms/step - accuracy: 0.9476 - loss: 0.3457 - val_accuracy: 0.9344 -
val_loss: 0.3889
Epoch 17/80
469/469  4s 8ms/step - accuracy: 0.9476 - loss: 0.3383 - val_accuracy: 0.9451 -
val_loss: 0.3576
Epoch 18/80
469/469  4s 8ms/step - accuracy: 0.9501 - loss: 0.3283 - val_accuracy: 0.9040 -
val_loss: 0.4642
Epoch 19/80
469/469  3s 7ms/step - accuracy: 0.9490 - loss: 0.3317 - val_accuracy: 0.9085 -
val_loss: 0.4708
Epoch 20/80
469/469  3s 7ms/step - accuracy: 0.9516 - loss: 0.3197 - val_accuracy: 0.9271 -
val_loss: 0.3918
Epoch 21/80
469/469  3s 7ms/step - accuracy: 0.9515 - loss: 0.3156 - val_accuracy: 0.9210 -
val_loss: 0.4088
Epoch 22/80
469/469  3s 7ms/step - accuracy: 0.9513 - loss: 0.3087 - val_accuracy: 0.9394 -
val_loss: 0.3589
Epoch 23/80
469/469  3s 7ms/step - accuracy: 0.9541 - loss: 0.3044 - val_accuracy: 0.9298 -
val_loss: 0.3795
Epoch 24/80
469/469  3s 7ms/step - accuracy: 0.9531 - loss: 0.3037 - val_accuracy: 0.9211 -
val_loss: 0.4236
Epoch 25/80
469/469  3s 7ms/step - accuracy: 0.9535 - loss: 0.3015 - val_accuracy: 0.9345 -
val_loss: 0.3546
Epoch 26/80
469/469  3s 7ms/step - accuracy: 0.9521 - loss: 0.3030 - val_accuracy: 0.9419 -
```

```

val_loss: 0.3367
Epoch 27/80
469/469 ————— 3s 7ms/step - accuracy: 0.9539 - loss: 0.2963 - val_accuracy: 0.9443 -
val_loss: 0.3294
Epoch 28/80
469/469 ————— 3s 7ms/step - accuracy: 0.9554 - loss: 0.2948 - val_accuracy: 0.9222 -
val_loss: 0.3912
Epoch 29/80
469/469 ————— 3s 7ms/step - accuracy: 0.9526 - loss: 0.2953 - val_accuracy: 0.9346 -
val_loss: 0.3664
Epoch 30/80
469/469 ————— 3s 7ms/step - accuracy: 0.9535 - loss: 0.2899 - val_accuracy: 0.9146 -
val_loss: 0.3918
Epoch 31/80
469/469 ————— 3s 7ms/step - accuracy: 0.9531 - loss: 0.2884 - val_accuracy: 0.9013 -
val_loss: 0.4499
Epoch 32/80
469/469 ————— 3s 7ms/step - accuracy: 0.9554 - loss: 0.2823 - val_accuracy: 0.9404 -
val_loss: 0.3314
Epoch 33/80
469/469 ————— 3s 7ms/step - accuracy: 0.9549 - loss: 0.2835 - val_accuracy: 0.9284 -
val_loss: 0.3719
Epoch 34/80
469/469 ————— 3s 7ms/step - accuracy: 0.9559 - loss: 0.2799 - val_accuracy: 0.9448 -
val_loss: 0.3088
Epoch 35/80
469/469 ————— 3s 7ms/step - accuracy: 0.9551 - loss: 0.2800 - val_accuracy: 0.9534 -
val_loss: 0.2879
Epoch 36/80
469/469 ————— 3s 7ms/step - accuracy: 0.9578 - loss: 0.2689 - val_accuracy: 0.9471 -
val_loss: 0.3148
Epoch 37/80
469/469 ————— 3s 7ms/step - accuracy: 0.9564 - loss: 0.2755 - val_accuracy: 0.9433 -
val_loss: 0.3170
Epoch 38/80
469/469 ————— 4s 7ms/step - accuracy: 0.9570 - loss: 0.2694 - val_accuracy: 0.9286 -
val_loss: 0.3812
Epoch 39/80
469/469 ————— 5s 7ms/step - accuracy: 0.9576 - loss: 0.2679 - val_accuracy: 0.9347 -

```

```

val_loss: 0.3234
Epoch 40/80
469/469 ————— 3s 7ms/step - accuracy: 0.9568 - loss: 0.2639 - val_accuracy: 0.9514 -
val_loss: 0.2787
Epoch 41/80
469/469 ————— 3s 7ms/step - accuracy: 0.9595 - loss: 0.2598 - val_accuracy: 0.9377 -
val_loss: 0.3187
Epoch 42/80
469/469 ————— 3s 7ms/step - accuracy: 0.9590 - loss: 0.2606 - val_accuracy: 0.9266 -
val_loss: 0.3755
Epoch 43/80
469/469 ————— 3s 7ms/step - accuracy: 0.9579 - loss: 0.2649 - val_accuracy: 0.9471 -
val_loss: 0.2963
Epoch 44/80
469/469 ————— 3s 7ms/step - accuracy: 0.9571 - loss: 0.2692 - val_accuracy: 0.9389 -
val_loss: 0.3156
Epoch 45/80
469/469 ————— 3s 7ms/step - accuracy: 0.9572 - loss: 0.2643 - val_accuracy: 0.9282 -
val_loss: 0.3606
Epoch 46/80
469/469 ————— 3s 7ms/step - accuracy: 0.9586 - loss: 0.2569 - val_accuracy: 0.9240 -
val_loss: 0.3671
Epoch 47/80
469/469 ————— 3s 7ms/step - accuracy: 0.9582 - loss: 0.2594 - val_accuracy: 0.9196 -
val_loss: 0.3642
Epoch 48/80
469/469 ————— 3s 7ms/step - accuracy: 0.9584 - loss: 0.2571 - val_accuracy: 0.9331 -
val_loss: 0.3373
Epoch 49/80
469/469 ————— 3s 7ms/step - accuracy: 0.9584 - loss: 0.2576 - val_accuracy: 0.9096 -
val_loss: 0.3980
Epoch 50/80
469/469 ————— 3s 7ms/step - accuracy: 0.9592 - loss: 0.2527 - val_accuracy: 0.9081 -
val_loss: 0.4122
Epoch 51/80
469/469 ————— 3s 7ms/step - accuracy: 0.9589 - loss: 0.2546 - val_accuracy: 0.9285 -
val_loss: 0.3577
Epoch 52/80
469/469 ————— 3s 7ms/step - accuracy: 0.9574 - loss: 0.2528 - val_accuracy: 0.9367 -

```

```
val_loss: 0.3179
Epoch 53/80
469/469 ————— 4s 8ms/step - accuracy: 0.9605 - loss: 0.2535 - val_accuracy: 0.9204 -
val_loss: 0.3596
Epoch 54/80
469/469 ————— 4s 8ms/step - accuracy: 0.9593 - loss: 0.2523 - val_accuracy: 0.9478 -
val_loss: 0.2983
Epoch 55/80
469/469 ————— 4s 8ms/step - accuracy: 0.9598 - loss: 0.2547 - val_accuracy: 0.9407 -
val_loss: 0.3177
Epoch 56/80
469/469 ————— 4s 8ms/step - accuracy: 0.9602 - loss: 0.2502 - val_accuracy: 0.9384 -
val_loss: 0.3272
Epoch 57/80
469/469 ————— 4s 8ms/step - accuracy: 0.9598 - loss: 0.2526 - val_accuracy: 0.9245 -
val_loss: 0.3522
Epoch 58/80
469/469 ————— 4s 8ms/step - accuracy: 0.9606 - loss: 0.2472 - val_accuracy: 0.9350 -
val_loss: 0.3354
Epoch 59/80
469/469 ————— 4s 8ms/step - accuracy: 0.9594 - loss: 0.2498 - val_accuracy: 0.9236 -
val_loss: 0.3554
Epoch 60/80
469/469 ————— 4s 8ms/step - accuracy: 0.9584 - loss: 0.2482 - val_accuracy: 0.9489 -
val_loss: 0.2794
Epoch 61/80
469/469 ————— 4s 8ms/step - accuracy: 0.9584 - loss: 0.2518 - val_accuracy: 0.8980 -
val_loss: 0.4484
Epoch 62/80
469/469 ————— 4s 8ms/step - accuracy: 0.9601 - loss: 0.2467 - val_accuracy: 0.9219 -
val_loss: 0.3713
Epoch 63/80
469/469 ————— 4s 8ms/step - accuracy: 0.9598 - loss: 0.2484 - val_accuracy: 0.9403 -
val_loss: 0.3067
Epoch 64/80
469/469 ————— 4s 8ms/step - accuracy: 0.9582 - loss: 0.2474 - val_accuracy: 0.9408 -
val_loss: 0.3110
Epoch 65/80
469/469 ————— 4s 8ms/step - accuracy: 0.9591 - loss: 0.2464 - val_accuracy: 0.8700 -
```

```

val_loss: 0.5445
Epoch 66/80
469/469 ————— 4s 8ms/step - accuracy: 0.9589 - loss: 0.2447 - val_accuracy: 0.9505 -
val_loss: 0.2735
Epoch 67/80
469/469 ————— 4s 8ms/step - accuracy: 0.9590 - loss: 0.2460 - val_accuracy: 0.8870 -
val_loss: 0.4848
Epoch 68/80
469/469 ————— 4s 8ms/step - accuracy: 0.9608 - loss: 0.2414 - val_accuracy: 0.9474 -
val_loss: 0.2810
Epoch 69/80
469/469 ————— 4s 8ms/step - accuracy: 0.9600 - loss: 0.2402 - val_accuracy: 0.9283 -
val_loss: 0.3549
Epoch 70/80
469/469 ————— 4s 8ms/step - accuracy: 0.9594 - loss: 0.2461 - val_accuracy: 0.9224 -
val_loss: 0.3638
Epoch 71/80
469/469 ————— 4s 8ms/step - accuracy: 0.9618 - loss: 0.2366 - val_accuracy: 0.9397 -
val_loss: 0.2981
Epoch 72/80
469/469 ————— 4s 8ms/step - accuracy: 0.9598 - loss: 0.2393 - val_accuracy: 0.9451 -
val_loss: 0.2818
Epoch 73/80
469/469 ————— 4s 8ms/step - accuracy: 0.9605 - loss: 0.2384 - val_accuracy: 0.9545 -
val_loss: 0.2616
Epoch 74/80
469/469 ————— 4s 9ms/step - accuracy: 0.9602 - loss: 0.2409 - val_accuracy: 0.9518 -
val_loss: 0.2689
Epoch 75/80
469/469 ————— 4s 9ms/step - accuracy: 0.9588 - loss: 0.2457 - val_accuracy: 0.9455 -
val_loss: 0.2915
Epoch 76/80
469/469 ————— 4s 8ms/step - accuracy: 0.9597 - loss: 0.2392 - val_accuracy: 0.8938 -
val_loss: 0.4367
Epoch 77/80
469/469 ————— 4s 8ms/step - accuracy: 0.9599 - loss: 0.2396 - val_accuracy: 0.9486 -
val_loss: 0.2748
Epoch 78/80
469/469 ————— 4s 8ms/step - accuracy: 0.9608 - loss: 0.2374 - val_accuracy: 0.9494 -

```

val_loss: 0.2865

Epoch 79/80

469/469 ————— 4s 8ms/step - accuracy: 0.9609 - loss: 0.2341 - val_accuracy: 0.9507 -

val_loss: 0.2675

Epoch 80/80

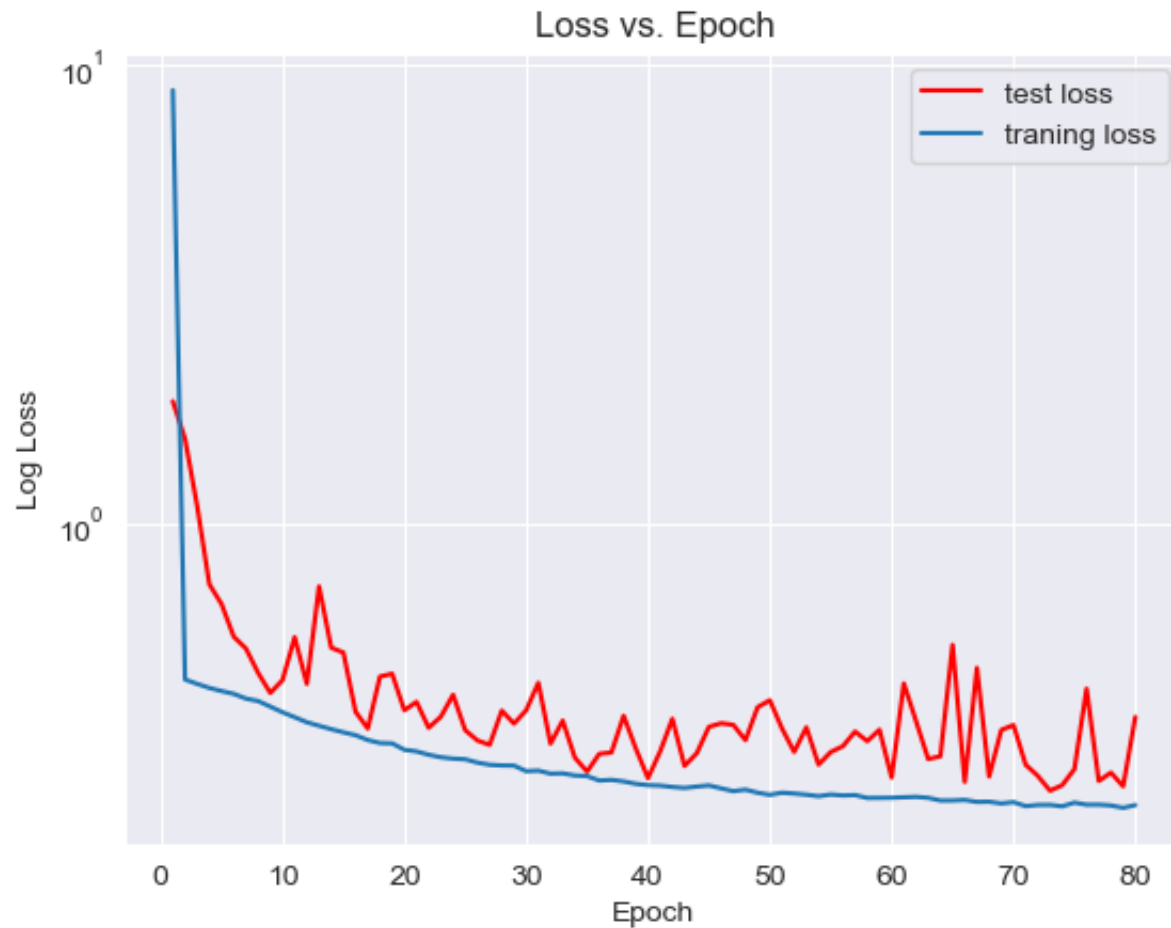
469/469 ————— 4s 8ms/step - accuracy: 0.9613 - loss: 0.2363 - val_accuracy: 0.9175 -

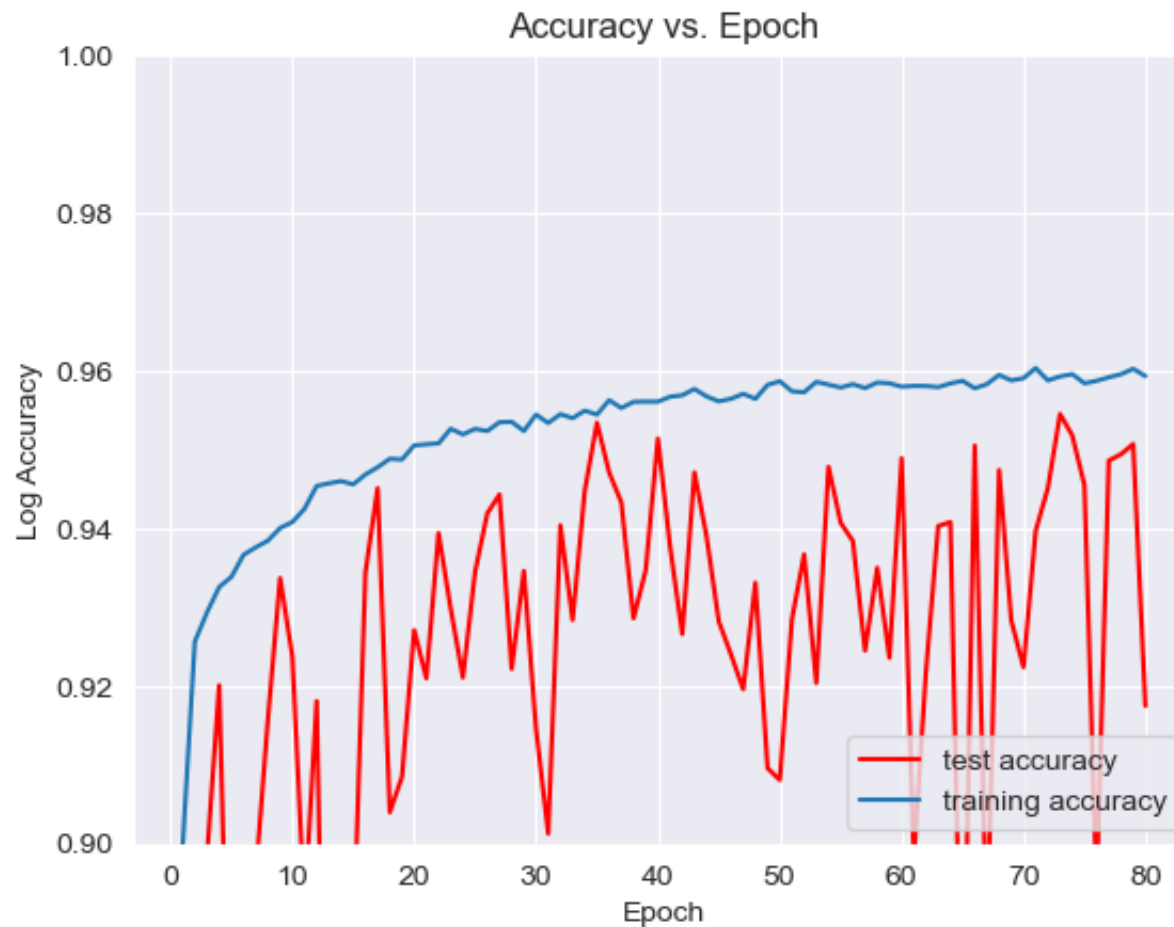
val_loss: 0.3788

313/313 ————— 1s 3ms/step - accuracy: 0.9103 - loss: 0.4072

Test accuracy = 0.918 Test loss = 0.379

```
In [25]: plot_loss(history_nnbr)
         plot_accuracy(history_nnbr)
```





? Compare the graphs to the train and test loss and train and test accuracy to the foregoing model without the batch normalization and answer these questions:

1. What does the difference in smoothness, especially for the test accuracy, tell you about the change in behavior of the gradient?
2. Notice the overall shape of the learning metric curves. What might the abrupt change in slope of the validation accuracy curve tell you about the learning rate?

End of exercise.

✓ Answers:

1. The smoothness actually decreased by almost an order of magnitude. The reason for this is the momentum value, which is very close to one. This causes a mismatch between inference and training, leading to differences in how the Dense layer outputs are scaled before activation in these two regimes. As a result, the model behaves differently during validation, negatively affecting its generalization.
2. The overall shape of the learning curve, which rapidly increases in the first few epochs followed by an abrupt flattening, is most likely caused by a learning rate that is too high.

7.0 Learning Rate Decay

The foregoing model training appears to have worked rather well. But, can we do better? One possibility is to use a **adaptive learning rate** or **decaying learning rate**. Keras, like most deep learning frameworks, provides [utilities to define decaying learning rate schedules](#). Here we will work with an exponential learning rate. You can see specific examples of using this utility [here](#).

Validation Accuracy	L2 regularization	Learning rate	Weight decay	Batch normalization
Erratic results	0.2	initial rate=5e-4	None	Momentum=0.998
Very slow learning	0.2	initial rate=5e-6	None	Momentum=0.998
.9225	0.02	initial rate=5e-5	0.01	Momentum=0.998
.9258	0.002	initial rate=5e-5	0.01	Momentum=0.998
.9640	0.002	initial rate=5e-5	0.001	Momentum=0.998
.9644	0.0002	initial rate=5e-5	0.001	Momentum=0.998
.9779	0.0002	initial rate=5e-5	0.0001	Momentum=0.998

You may well wonder why the introduction of an exponentially decaying learning rate schedule requires such drastic reduction in other regularization hyperparameter values. This is not an unusual situation. As we increase one kind of

constraint on learning, it is often necessary to decrease others, or else the learning becomes too slow or can even stall.

► **Exercise 5-9:** You will now implement and test a model with an exponentially decreasing learning rate:

1. Start with the model specification you used for Exercise 5-8, change the model name to `nnvr`.
2. For the dense hidden layer set the L2 regularization hyperparameter to 0.0002.
3. Create a learning rate schedule using the [Keras exponential learning rate scheduler] (https://keras.io/api/optimizers/learning_rate_schedules/exponential_decay/) with 'initial_learning_rate=5e-5', 'decay_steps=1000', 'decay_rate=0.9'.
4. Compile the model with the optimizer=RMSprop(learning_rate=your_learning_rate_schedule, weight_decay=0.0001).
5. Fit the model for 80 epochs, saving the history object. This will take some time as before.
6. Plot the training and test loss and accuracy.

In [26]: *## Your code to define the model and display the summary goes here*

```
nnvr = models.Sequential(name='Exercise-5-8')

nnvr.add(layers.Input((28*28,)))
nnvr.add(layers.Dense(512, kernel_regularizer=regularizers.l2(0.0002)))
nnvr.add(layers.BatchNormalization(momentum=0.998))
nnvr.add(layers.Activation('relu'))
nnvr.add(layers.Dense(10, activation='softmax'))

lr_sched = keras.optimizers.schedules.ExponentialDecay(
    5e-5,
    decay_steps=1000,
    decay_rate=0.9
)

opt = RMSprop(
    learning_rate=lr_sched,
```

```

    weight_decay=0.0001
)

nnvr.compile(
    optimizer=opt,
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_nnvr = nnvr.fit(
    train_images,
    train_labels,
    validation_data=(test_images, test_labels),
    epochs=80,
    batch_size=128
);

test_loss, test_accuracy = nnvr.evaluate(test_images, test_labels)
print("Test accuracy = {0:.3f}    Test loss = {1:.3f}".format(test_accuracy, test_loss))

```

Epoch 1/80

469/469  **5s** 8ms/step - accuracy: 0.6922 - loss: 1.1602 - val_accuracy: 0.9029 - val_loss: 0.8623

Epoch 2/80

469/469  **4s** 7ms/step - accuracy: 0.9288 - loss: 0.3866 - val_accuracy: 0.9420 - val_loss: 0.4648

Epoch 3/80

469/469  **5s** 10ms/step - accuracy: 0.9449 - loss: 0.3200 - val_accuracy: 0.9514 - val_loss: 0.3300

Epoch 4/80

469/469  **4s** 8ms/step - accuracy: 0.9530 - loss: 0.2862 - val_accuracy: 0.9565 - val_loss: 0.2799

Epoch 5/80

469/469  **3s** 7ms/step - accuracy: 0.9588 - loss: 0.2640 - val_accuracy: 0.9604 - val_loss: 0.2547

Epoch 6/80

469/469  **3s** 7ms/step - accuracy: 0.9624 - loss: 0.2482 - val_accuracy: 0.9622 - val_loss: 0.2391

Epoch 7/80

469/469  **3s** 7ms/step - accuracy: 0.9669 - loss: 0.2303 - val_accuracy: 0.9645 -

```

val_loss: 0.2284
Epoch 8/80
469/469 ————— 4s 9ms/step - accuracy: 0.9687 - loss: 0.2223 - val_accuracy: 0.9657 -
val_loss: 0.2203
Epoch 9/80
469/469 ————— 4s 7ms/step - accuracy: 0.9711 - loss: 0.2102 - val_accuracy: 0.9669 -
val_loss: 0.2125
Epoch 10/80
469/469 ————— 3s 7ms/step - accuracy: 0.9726 - loss: 0.2035 - val_accuracy: 0.9684 -
val_loss: 0.2066
Epoch 11/80
469/469 ————— 3s 7ms/step - accuracy: 0.9758 - loss: 0.1925 - val_accuracy: 0.9704 -
val_loss: 0.2013
Epoch 12/80
469/469 ————— 3s 7ms/step - accuracy: 0.9764 - loss: 0.1877 - val_accuracy: 0.9699 -
val_loss: 0.1966
Epoch 13/80
469/469 ————— 4s 8ms/step - accuracy: 0.9785 - loss: 0.1795 - val_accuracy: 0.9708 -
val_loss: 0.1918
Epoch 14/80
469/469 ————— 4s 7ms/step - accuracy: 0.9794 - loss: 0.1750 - val_accuracy: 0.9716 -
val_loss: 0.1887
Epoch 15/80
469/469 ————— 3s 7ms/step - accuracy: 0.9796 - loss: 0.1701 - val_accuracy: 0.9717 -
val_loss: 0.1845
Epoch 16/80
469/469 ————— 3s 7ms/step - accuracy: 0.9814 - loss: 0.1662 - val_accuracy: 0.9727 -
val_loss: 0.1813
Epoch 17/80
469/469 ————— 4s 7ms/step - accuracy: 0.9824 - loss: 0.1607 - val_accuracy: 0.9733 -
val_loss: 0.1784
Epoch 18/80
469/469 ————— 3s 7ms/step - accuracy: 0.9815 - loss: 0.1602 - val_accuracy: 0.9738 -
val_loss: 0.1757
Epoch 19/80
469/469 ————— 4s 7ms/step - accuracy: 0.9830 - loss: 0.1545 - val_accuracy: 0.9739 -
val_loss: 0.1729
Epoch 20/80
469/469 ————— 4s 7ms/step - accuracy: 0.9836 - loss: 0.1520 - val_accuracy: 0.9733 -

```

```
val_loss: 0.1707
Epoch 21/80
469/469 ————— 4s 7ms/step - accuracy: 0.9842 - loss: 0.1473 - val_accuracy: 0.9747 -
val_loss: 0.1688
Epoch 22/80
469/469 ————— 4s 8ms/step - accuracy: 0.9857 - loss: 0.1439 - val_accuracy: 0.9744 -
val_loss: 0.1669
Epoch 23/80
469/469 ————— 3s 7ms/step - accuracy: 0.9851 - loss: 0.1446 - val_accuracy: 0.9747 -
val_loss: 0.1648
Epoch 24/80
469/469 ————— 4s 8ms/step - accuracy: 0.9861 - loss: 0.1411 - val_accuracy: 0.9750 -
val_loss: 0.1635
Epoch 25/80
469/469 ————— 4s 8ms/step - accuracy: 0.9866 - loss: 0.1377 - val_accuracy: 0.9749 -
val_loss: 0.1618
Epoch 26/80
469/469 ————— 4s 8ms/step - accuracy: 0.9869 - loss: 0.1353 - val_accuracy: 0.9753 -
val_loss: 0.1603
Epoch 27/80
469/469 ————— 4s 8ms/step - accuracy: 0.9870 - loss: 0.1347 - val_accuracy: 0.9759 -
val_loss: 0.1590
Epoch 28/80
469/469 ————— 4s 8ms/step - accuracy: 0.9871 - loss: 0.1327 - val_accuracy: 0.9763 -
val_loss: 0.1577
Epoch 29/80
469/469 ————— 4s 8ms/step - accuracy: 0.9875 - loss: 0.1328 - val_accuracy: 0.9761 -
val_loss: 0.1567
Epoch 30/80
469/469 ————— 4s 8ms/step - accuracy: 0.9883 - loss: 0.1289 - val_accuracy: 0.9763 -
val_loss: 0.1560
Epoch 31/80
469/469 ————— 4s 8ms/step - accuracy: 0.9881 - loss: 0.1282 - val_accuracy: 0.9766 -
val_loss: 0.1550
Epoch 32/80
469/469 ————— 4s 8ms/step - accuracy: 0.9890 - loss: 0.1267 - val_accuracy: 0.9765 -
val_loss: 0.1538
Epoch 33/80
469/469 ————— 4s 8ms/step - accuracy: 0.9887 - loss: 0.1250 - val_accuracy: 0.9763 -
```

```
val_loss: 0.1529
Epoch 34/80
469/469 ————— 4s 8ms/step - accuracy: 0.9892 - loss: 0.1247 - val_accuracy: 0.9764 -
val_loss: 0.1520
Epoch 35/80
469/469 ————— 4s 8ms/step - accuracy: 0.9902 - loss: 0.1218 - val_accuracy: 0.9764 -
val_loss: 0.1513
Epoch 36/80
469/469 ————— 4s 8ms/step - accuracy: 0.9892 - loss: 0.1227 - val_accuracy: 0.9765 -
val_loss: 0.1506
Epoch 37/80
469/469 ————— 4s 8ms/step - accuracy: 0.9896 - loss: 0.1222 - val_accuracy: 0.9765 -
val_loss: 0.1500
Epoch 38/80
469/469 ————— 4s 8ms/step - accuracy: 0.9900 - loss: 0.1195 - val_accuracy: 0.9766 -
val_loss: 0.1494
Epoch 39/80
469/469 ————— 4s 8ms/step - accuracy: 0.9895 - loss: 0.1208 - val_accuracy: 0.9766 -
val_loss: 0.1487
Epoch 40/80
469/469 ————— 4s 8ms/step - accuracy: 0.9909 - loss: 0.1174 - val_accuracy: 0.9766 -
val_loss: 0.1481
Epoch 41/80
469/469 ————— 4s 8ms/step - accuracy: 0.9898 - loss: 0.1181 - val_accuracy: 0.9768 -
val_loss: 0.1475
Epoch 42/80
469/469 ————— 4s 8ms/step - accuracy: 0.9907 - loss: 0.1159 - val_accuracy: 0.9770 -
val_loss: 0.1470
Epoch 43/80
469/469 ————— 4s 8ms/step - accuracy: 0.9911 - loss: 0.1148 - val_accuracy: 0.9769 -
val_loss: 0.1466
Epoch 44/80
469/469 ————— 4s 8ms/step - accuracy: 0.9903 - loss: 0.1154 - val_accuracy: 0.9775 -
val_loss: 0.1461
Epoch 45/80
469/469 ————— 4s 8ms/step - accuracy: 0.9912 - loss: 0.1144 - val_accuracy: 0.9770 -
val_loss: 0.1456
Epoch 46/80
469/469 ————— 4s 8ms/step - accuracy: 0.9911 - loss: 0.1145 - val_accuracy: 0.9774 -
```

```

val_loss: 0.1453
Epoch 47/80
469/469 ————— 4s 8ms/step - accuracy: 0.9908 - loss: 0.1139 - val_accuracy: 0.9773 -
val_loss: 0.1448
Epoch 48/80
469/469 ————— 4s 9ms/step - accuracy: 0.9908 - loss: 0.1125 - val_accuracy: 0.9774 -
val_loss: 0.1444
Epoch 49/80
469/469 ————— 4s 8ms/step - accuracy: 0.9917 - loss: 0.1118 - val_accuracy: 0.9775 -
val_loss: 0.1440
Epoch 50/80
469/469 ————— 4s 8ms/step - accuracy: 0.9913 - loss: 0.1126 - val_accuracy: 0.9775 -
val_loss: 0.1438
Epoch 51/80
469/469 ————— 4s 8ms/step - accuracy: 0.9907 - loss: 0.1124 - val_accuracy: 0.9776 -
val_loss: 0.1434
Epoch 52/80
469/469 ————— 4s 8ms/step - accuracy: 0.9918 - loss: 0.1102 - val_accuracy: 0.9776 -
val_loss: 0.1431
Epoch 53/80
469/469 ————— 5s 9ms/step - accuracy: 0.9916 - loss: 0.1094 - val_accuracy: 0.9777 -
val_loss: 0.1429
Epoch 54/80
469/469 ————— 4s 8ms/step - accuracy: 0.9922 - loss: 0.1088 - val_accuracy: 0.9778 -
val_loss: 0.1426
Epoch 55/80
469/469 ————— 4s 8ms/step - accuracy: 0.9920 - loss: 0.1098 - val_accuracy: 0.9776 -
val_loss: 0.1424
Epoch 56/80
469/469 ————— 4s 9ms/step - accuracy: 0.9913 - loss: 0.1098 - val_accuracy: 0.9777 -
val_loss: 0.1422
Epoch 57/80
469/469 ————— 5s 10ms/step - accuracy: 0.9917 - loss: 0.1080 - val_accuracy: 0.9777
- val_loss: 0.1419
Epoch 58/80
469/469 ————— 4s 8ms/step - accuracy: 0.9922 - loss: 0.1082 - val_accuracy: 0.9776 -
val_loss: 0.1417
Epoch 59/80
469/469 ————— 5s 7ms/step - accuracy: 0.9924 - loss: 0.1077 - val_accuracy: 0.9776 -

```



```

val_loss: 0.1415
Epoch 60/80
469/469 ————— 4s 8ms/step - accuracy: 0.9918 - loss: 0.1088 - val_accuracy: 0.9775 -
val_loss: 0.1413
Epoch 61/80
469/469 ————— 4s 8ms/step - accuracy: 0.9925 - loss: 0.1075 - val_accuracy: 0.9776 -
val_loss: 0.1412
Epoch 62/80
469/469 ————— 4s 8ms/step - accuracy: 0.9919 - loss: 0.1070 - val_accuracy: 0.9777 -
val_loss: 0.1410
Epoch 63/80
469/469 ————— 4s 8ms/step - accuracy: 0.9924 - loss: 0.1071 - val_accuracy: 0.9778 -
val_loss: 0.1408
Epoch 64/80
469/469 ————— 4s 8ms/step - accuracy: 0.9926 - loss: 0.1068 - val_accuracy: 0.9776 -
val_loss: 0.1407
Epoch 65/80
469/469 ————— 4s 8ms/step - accuracy: 0.9924 - loss: 0.1062 - val_accuracy: 0.9777 -
val_loss: 0.1405
Epoch 66/80
469/469 ————— 4s 8ms/step - accuracy: 0.9923 - loss: 0.1054 - val_accuracy: 0.9779 -
val_loss: 0.1404
Epoch 67/80
469/469 ————— 4s 8ms/step - accuracy: 0.9923 - loss: 0.1058 - val_accuracy: 0.9778 -
val_loss: 0.1403
Epoch 68/80
469/469 ————— 4s 8ms/step - accuracy: 0.9923 - loss: 0.1067 - val_accuracy: 0.9778 -
val_loss: 0.1402
Epoch 69/80
469/469 ————— 4s 8ms/step - accuracy: 0.9925 - loss: 0.1057 - val_accuracy: 0.9780 -
val_loss: 0.1401
Epoch 70/80
469/469 ————— 4s 8ms/step - accuracy: 0.9921 - loss: 0.1059 - val_accuracy: 0.9778 -
val_loss: 0.1399
Epoch 71/80
469/469 ————— 4s 8ms/step - accuracy: 0.9925 - loss: 0.1064 - val_accuracy: 0.9777 -
val_loss: 0.1398
Epoch 72/80
469/469 ————— 4s 8ms/step - accuracy: 0.9928 - loss: 0.1051 - val_accuracy: 0.9777 -

```

```

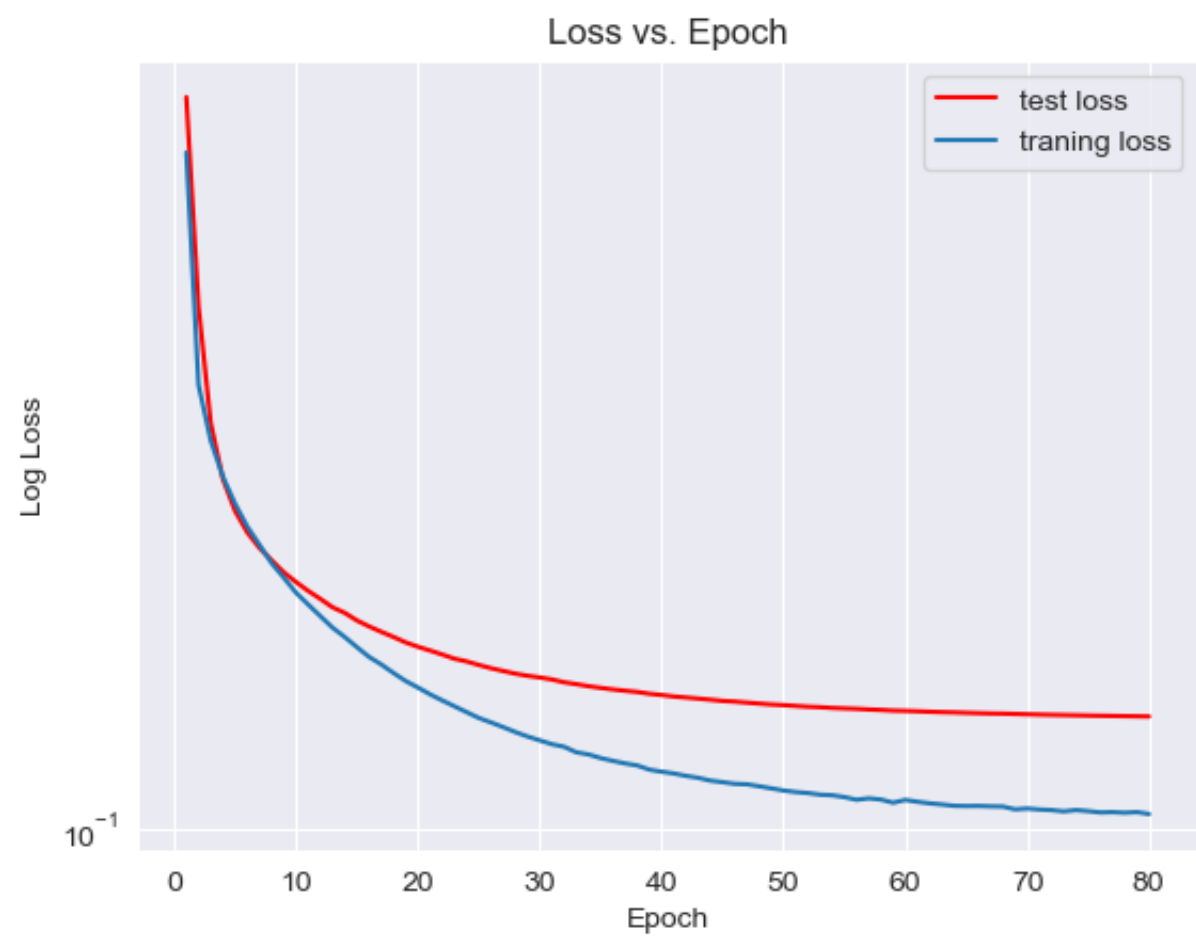
val_loss: 0.1397
Epoch 73/80
469/469 ————— 4s 8ms/step - accuracy: 0.9924 - loss: 0.1060 - val_accuracy: 0.9778 -
val_loss: 0.1396
Epoch 74/80
469/469 ————— 5s 8ms/step - accuracy: 0.9928 - loss: 0.1037 - val_accuracy: 0.9778 -
val_loss: 0.1395
Epoch 75/80
469/469 ————— 4s 8ms/step - accuracy: 0.9920 - loss: 0.1066 - val_accuracy: 0.9779 -
val_loss: 0.1394
Epoch 76/80
469/469 ————— 4s 8ms/step - accuracy: 0.9919 - loss: 0.1055 - val_accuracy: 0.9780 -
val_loss: 0.1393
Epoch 77/80
469/469 ————— 4s 8ms/step - accuracy: 0.9927 - loss: 0.1044 - val_accuracy: 0.9779 -
val_loss: 0.1393
Epoch 78/80
469/469 ————— 4s 8ms/step - accuracy: 0.9929 - loss: 0.1047 - val_accuracy: 0.9779 -
val_loss: 0.1392
Epoch 79/80
469/469 ————— 4s 8ms/step - accuracy: 0.9930 - loss: 0.1039 - val_accuracy: 0.9778 -
val_loss: 0.1391
Epoch 80/80
469/469 ————— 4s 8ms/step - accuracy: 0.9931 - loss: 0.1037 - val_accuracy: 0.9779 -
val_loss: 0.1390
313/313 ————— 1s 3ms/step - accuracy: 0.9732 - loss: 0.1554
Test accuracy = 0.978   Test loss = 0.139

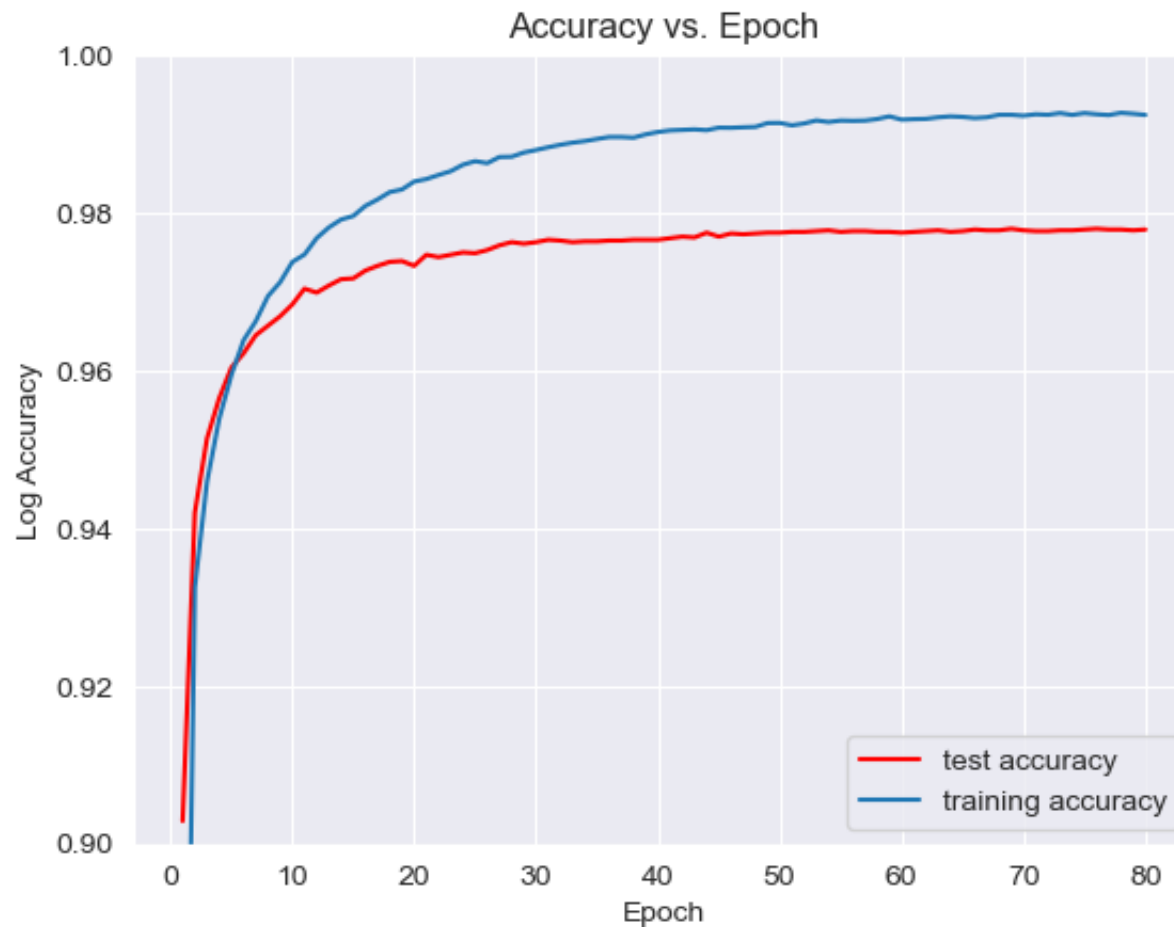
```

```

In [27]: plot_loss(history_nnvr)
         plot_accuracy(history_nnvr)

```





```
In [28]: test_loss, test_accuracy = nnvr.evaluate(test_images, test_labels)
print("Test accuracy = {0:.3f}   Test loss = {1:.3f}".format(test_accuracy, test_loss))
```

313/313 ————— 1s 2ms/step — accuracy: 0.9732 — loss: 0.1554
 Test accuracy = 0.978 Test loss = 0.139

? Examine the results and answer these questions:

1. Compare the validation accuracy curve and final accuracy to the same curve and final accuracy value for the model trained with fixed learning rate. What do the differences in the curve and final learning rate tell you about the effect of using a variable learning rate?

2. Examine the shape of the training and validation curves. What does this shape tell you about the learning of this model?

End of exercise.

 Answers:

1. The exponentially decaying learning rate model has higher accuracies compared to the fixed learning rate model, with a final accuracy of approximately 97.3% vs. 96.1% and a validation accuracy of 97.8% vs. 95%.
2. The difference between the fixed learning rate and the exponentially decaying learning rate is worth noticing. The latter demonstrates very stable and smooth learning curves for both training and validation, indicating significantly lower variance compared to the fixed learning rate model and higher stability. Additionally, the exponentially decaying learning rate curve suggests higher overall performance.

8.0 Dropout Regularization

Dropout regularization randomly drops (set to zero weight) connections in the neural network. The fraction of weights dropped is a hyperparameter which must be set. A good typical starting values is around 0.5. Emerical evidence indicates that results are not particularly sensitive to this value.

Here, the batch normalization layer has been replaced by a dropout regularization layer. This seemingly small change actually has a large effect on the model architecture. As a result, an extensive hyperparameter search was required to achieve good model performance. The hyperparameter choices tried are summarized in the table below.

You many wonder why the final model selected has lower accuacy than some other hyperparameter choices. These seemingly superior choices resulted in very odd convergence behavior of the model. In these cases the training metrics continued to improve considerably, whereas the validation metrics where stuck at a significantly different level. This outcome indicated that the model was in overfit or the training was stuck at some local optimum. To correct this problem, additional mild regularizaton was added. The regularization added bias to the model training,

reducing the accuracy metric slightly, but corrected the problem of serious over-fitting.

Important note: For the simplicity of our examples, we have substituted the dropout regularization layer for the batch normalization layer. You should not conclude from this example that one cannot or should not use both types of regularizers. In fact, in many deep architectures using both dropout regularization and batch normalization together is common practice.

Validation Accuracy	L2 regularization	Learning rate	Weight decay	Dropout rate
Very slow learning	0.2	initial rate=5e-5	None	0.5
Very slow learning	0.02	initial rate=5e-5	None	0.5
Very slow learning	None	initial rate=5e-5	0.01	0.5
.9525	None	initial rate=5e-5	0.0001	0.5
.9620	None	initial rate=5e-5	none	0.5
.9666	None	initial rate=5e-5	none	0.3
.9835, divergence between train and validation	None	initial rate=5e-4	none	0.3
.9832, divergence between train and validation	None	initial rate=5e-4	none	0.5
.9838, divergence between train and validation	None	initial rate=5e-4	none	0.1
.9822, less divergence	0.0002	inittheial rate=5e-4	0.0001	0.1
.9814	0.0002	initial rate=5e-4	0.0001	0.1

► **Exercise 5-10:** You will now create and evaluate a model using a dropout regularization layer:

1. Start with the model specification you used for Exercise 5-9, change the model name to `nndo`.
2. For the dense hidden layer set the L2 regularization hypterparameter to 0.0002.
3. Replace the batch normalization layer with a [Dropout regularization layer](#) with `rate=0.1`.
4. Create a learning rate schedule using the [Keras exponential learning rate scheduler]

(https://keras.io/api/optimizers/learning_rate_schedules/exponential_decay/) with
'initial_learning_rate=5e-4', 'decay_steps=1000', 'decay_rate=0.9'.

5. Compile the model with the optimizer=RMSprop(learning_rate=lr_schedule, weight_decay=0.0001), argument.
6. Fit the model for 80 epochs, saving the history object. This will take some time as before.
7. Plot the training and test loss and accuracy.

In [29]: *## Your code goes here*

```
nndo = models.Sequential(name='Exercise-5-8')

nndo.add(layers.Input((28*28,)))
nndo.add(layers.Dense(512, kernel_regularizer=regularizers.l2(0.0002), activation='relu'))
nndo.add(layers.Dropout(0.1))
nndo.add(layers.Dense(10, activation='softmax'))

lr_sched = keras.optimizers.schedules.ExponentialDecay(
    5e-4,
    decay_steps=1000,
    decay_rate=0.9
)

opt = RMSprop(
    learning_rate=lr_sched,
    weight_decay=0.0001
)

nndo.compile(
    optimizer=opt,
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

history_nndo = nndo.fit(
    train_images,
    train_labels,
    validation_data=(test_images, test_labels),
```

```

    epochs=80,
    batch_size=128
);

test_loss, test_accuracy = nndo.evaluate(test_images, test_labels)
print("Test accuracy = {0:.3f}    Test loss = {1:.3f}".format(test_accuracy, test_loss))

```

Epoch 1/80

469/469 ————— 3s 6ms/step - accuracy: 0.8544 - loss: 0.6703 - val_accuracy: 0.9450 - val_loss: 0.2914

Epoch 2/80

469/469 ————— 3s 6ms/step - accuracy: 0.9497 - loss: 0.2754 - val_accuracy: 0.9630 - val_loss: 0.2117

Epoch 3/80

469/469 ————— 3s 7ms/step - accuracy: 0.9635 - loss: 0.2049 - val_accuracy: 0.9690 - val_loss: 0.1748

Epoch 4/80

469/469 ————— 3s 7ms/step - accuracy: 0.9737 - loss: 0.1604 - val_accuracy: 0.9724 - val_loss: 0.1529

Epoch 5/80

469/469 ————— 4s 8ms/step - accuracy: 0.9763 - loss: 0.1420 - val_accuracy: 0.9759 - val_loss: 0.1371

Epoch 6/80

469/469 ————— 4s 7ms/step - accuracy: 0.9798 - loss: 0.1258 - val_accuracy: 0.9775 - val_loss: 0.1262

Epoch 7/80

469/469 ————— 3s 7ms/step - accuracy: 0.9827 - loss: 0.1113 - val_accuracy: 0.9781 - val_loss: 0.1247

Epoch 8/80

469/469 ————— 4s 8ms/step - accuracy: 0.9844 - loss: 0.1010 - val_accuracy: 0.9780 - val_loss: 0.1134

Epoch 9/80

469/469 ————— 4s 8ms/step - accuracy: 0.9861 - loss: 0.0936 - val_accuracy: 0.9792 - val_loss: 0.1080

Epoch 10/80

469/469 ————— 3s 7ms/step - accuracy: 0.9885 - loss: 0.0854 - val_accuracy: 0.9802 - val_loss: 0.1037

Epoch 11/80

469/469 ————— 3s 7ms/step - accuracy: 0.9889 - loss: 0.0814 - val_accuracy: 0.9807 - val_loss: 0.1008

Epoch 12/80
469/469 ————— 3s 7ms/step - accuracy: 0.9897 - loss: 0.0777 - val_accuracy: 0.9797 - val_loss: 0.0992

Epoch 13/80
469/469 ————— 3s 7ms/step - accuracy: 0.9894 - loss: 0.0751 - val_accuracy: 0.9800 - val_loss: 0.0987

Epoch 14/80
469/469 ————— 3s 6ms/step - accuracy: 0.9915 - loss: 0.0696 - val_accuracy: 0.9797 - val_loss: 0.0941

Epoch 15/80
469/469 ————— 3s 6ms/step - accuracy: 0.9915 - loss: 0.0681 - val_accuracy: 0.9826 - val_loss: 0.0909

Epoch 16/80
469/469 ————— 5s 6ms/step - accuracy: 0.9926 - loss: 0.0652 - val_accuracy: 0.9814 - val_loss: 0.0903

Epoch 17/80
469/469 ————— 3s 6ms/step - accuracy: 0.9926 - loss: 0.0621 - val_accuracy: 0.9813 - val_loss: 0.0894

Epoch 18/80
469/469 ————— 3s 7ms/step - accuracy: 0.9930 - loss: 0.0605 - val_accuracy: 0.9821 - val_loss: 0.0870

Epoch 19/80
469/469 ————— 3s 7ms/step - accuracy: 0.9925 - loss: 0.0615 - val_accuracy: 0.9830 - val_loss: 0.0849

Epoch 20/80
469/469 ————— 4s 8ms/step - accuracy: 0.9936 - loss: 0.0586 - val_accuracy: 0.9811 - val_loss: 0.0866

Epoch 21/80
469/469 ————— 3s 6ms/step - accuracy: 0.9943 - loss: 0.0552 - val_accuracy: 0.9820 - val_loss: 0.0842

Epoch 22/80
469/469 ————— 3s 6ms/step - accuracy: 0.9948 - loss: 0.0537 - val_accuracy: 0.9815 - val_loss: 0.0836

Epoch 23/80
469/469 ————— 3s 6ms/step - accuracy: 0.9951 - loss: 0.0525 - val_accuracy: 0.9821 - val_loss: 0.0815

Epoch 24/80
469/469 ————— 3s 6ms/step - accuracy: 0.9951 - loss: 0.0515 - val_accuracy: 0.9822 - val_loss: 0.0815

Epoch 25/80
469/469 ————— 3s 7ms/step - accuracy: 0.9947 - loss: 0.0508 - val_accuracy: 0.9823 - val_loss: 0.0824
Epoch 26/80
469/469 ————— 4s 8ms/step - accuracy: 0.9951 - loss: 0.0500 - val_accuracy: 0.9826 - val_loss: 0.0804
Epoch 27/80
469/469 ————— 3s 7ms/step - accuracy: 0.9960 - loss: 0.0480 - val_accuracy: 0.9810 - val_loss: 0.0828
Epoch 28/80
469/469 ————— 4s 8ms/step - accuracy: 0.9955 - loss: 0.0479 - val_accuracy: 0.9832 - val_loss: 0.0794
Epoch 29/80
469/469 ————— 4s 8ms/step - accuracy: 0.9964 - loss: 0.0461 - val_accuracy: 0.9820 - val_loss: 0.0786
Epoch 30/80
469/469 ————— 4s 8ms/step - accuracy: 0.9960 - loss: 0.0457 - val_accuracy: 0.9822 - val_loss: 0.0780
Epoch 31/80
469/469 ————— 4s 8ms/step - accuracy: 0.9957 - loss: 0.0462 - val_accuracy: 0.9826 - val_loss: 0.0789
Epoch 32/80
469/469 ————— 3s 6ms/step - accuracy: 0.9962 - loss: 0.0451 - val_accuracy: 0.9832 - val_loss: 0.0774
Epoch 33/80
469/469 ————— 3s 6ms/step - accuracy: 0.9962 - loss: 0.0438 - val_accuracy: 0.9829 - val_loss: 0.0769
Epoch 34/80
469/469 ————— 3s 6ms/step - accuracy: 0.9963 - loss: 0.0437 - val_accuracy: 0.9821 - val_loss: 0.0760
Epoch 35/80
469/469 ————— 3s 6ms/step - accuracy: 0.9966 - loss: 0.0428 - val_accuracy: 0.9823 - val_loss: 0.0776
Epoch 36/80
469/469 ————— 3s 7ms/step - accuracy: 0.9963 - loss: 0.0433 - val_accuracy: 0.9825 - val_loss: 0.0759
Epoch 37/80
469/469 ————— 3s 6ms/step - accuracy: 0.9965 - loss: 0.0425 - val_accuracy: 0.9826 - val_loss: 0.0752

```

Epoch 38/80
469/469  3s 6ms/step - accuracy: 0.9968 - loss: 0.0419 - val_accuracy: 0.9830 -
val_loss: 0.0758
Epoch 39/80
469/469  3s 6ms/step - accuracy: 0.9972 - loss: 0.0410 - val_accuracy: 0.9829 -
val_loss: 0.0755
Epoch 40/80
469/469  3s 7ms/step - accuracy: 0.9972 - loss: 0.0403 - val_accuracy: 0.9829 -
val_loss: 0.0744
Epoch 41/80
469/469  4s 8ms/step - accuracy: 0.9971 - loss: 0.0405 - val_accuracy: 0.9824 -
val_loss: 0.0752
Epoch 42/80
469/469  4s 8ms/step - accuracy: 0.9970 - loss: 0.0404 - val_accuracy: 0.9832 -
val_loss: 0.0744
Epoch 43/80
469/469  4s 8ms/step - accuracy: 0.9973 - loss: 0.0395 - val_accuracy: 0.9824 -
val_loss: 0.0744
Epoch 44/80
469/469  4s 8ms/step - accuracy: 0.9971 - loss: 0.0401 - val_accuracy: 0.9826 -
val_loss: 0.0746
Epoch 45/80
469/469  4s 8ms/step - accuracy: 0.9974 - loss: 0.0397 - val_accuracy: 0.9825 -
val_loss: 0.0738
Epoch 46/80
469/469  4s 8ms/step - accuracy: 0.9973 - loss: 0.0393 - val_accuracy: 0.9819 -
val_loss: 0.0747
Epoch 47/80
469/469  4s 8ms/step - accuracy: 0.9971 - loss: 0.0390 - val_accuracy: 0.9827 -
val_loss: 0.0732
Epoch 48/80
469/469  4s 8ms/step - accuracy: 0.9972 - loss: 0.0387 - val_accuracy: 0.9830 -
val_loss: 0.0738
Epoch 49/80
469/469  4s 8ms/step - accuracy: 0.9968 - loss: 0.0391 - val_accuracy: 0.9827 -
val_loss: 0.0737
Epoch 50/80
469/469  4s 8ms/step - accuracy: 0.9966 - loss: 0.0397 - val_accuracy: 0.9826 -
val_loss: 0.0740

```

Epoch 51/80
469/469  **4s** 8ms/step - accuracy: 0.9977 - loss: 0.0380 - val_accuracy: 0.9831 - val_loss: 0.0730

Epoch 52/80
469/469  **4s** 9ms/step - accuracy: 0.9972 - loss: 0.0379 - val_accuracy: 0.9827 - val_loss: 0.0725

Epoch 53/80
469/469  **4s** 8ms/step - accuracy: 0.9976 - loss: 0.0373 - val_accuracy: 0.9830 - val_loss: 0.0727

Epoch 54/80
469/469  **4s** 8ms/step - accuracy: 0.9979 - loss: 0.0369 - val_accuracy: 0.9829 - val_loss: 0.0729

Epoch 55/80
469/469  **4s** 8ms/step - accuracy: 0.9980 - loss: 0.0365 - val_accuracy: 0.9831 - val_loss: 0.0727

Epoch 56/80
469/469  **5s** 8ms/step - accuracy: 0.9979 - loss: 0.0370 - val_accuracy: 0.9827 - val_loss: 0.0722

Epoch 57/80
469/469  **4s** 7ms/step - accuracy: 0.9975 - loss: 0.0374 - val_accuracy: 0.9831 - val_loss: 0.0719

Epoch 58/80
469/469  **4s** 8ms/step - accuracy: 0.9975 - loss: 0.0370 - val_accuracy: 0.9828 - val_loss: 0.0726

Epoch 59/80
469/469  **4s** 8ms/step - accuracy: 0.9978 - loss: 0.0367 - val_accuracy: 0.9824 - val_loss: 0.0729

Epoch 60/80
469/469  **4s** 8ms/step - accuracy: 0.9977 - loss: 0.0367 - val_accuracy: 0.9825 - val_loss: 0.0723

Epoch 61/80
469/469  **4s** 8ms/step - accuracy: 0.9979 - loss: 0.0364 - val_accuracy: 0.9825 - val_loss: 0.0716

Epoch 62/80
469/469  **4s** 9ms/step - accuracy: 0.9977 - loss: 0.0367 - val_accuracy: 0.9825 - val_loss: 0.0722

Epoch 63/80
469/469  **4s** 8ms/step - accuracy: 0.9978 - loss: 0.0360 - val_accuracy: 0.9827 - val_loss: 0.0716

Epoch 64/80
469/469 ————— 4s 9ms/step - accuracy: 0.9979 - loss: 0.0365 - val_accuracy: 0.9828 - val_loss: 0.0718
Epoch 65/80
469/469 ————— 4s 8ms/step - accuracy: 0.9978 - loss: 0.0362 - val_accuracy: 0.9828 - val_loss: 0.0719
Epoch 66/80
469/469 ————— 5s 8ms/step - accuracy: 0.9977 - loss: 0.0363 - val_accuracy: 0.9827 - val_loss: 0.0717
Epoch 67/80
469/469 ————— 4s 8ms/step - accuracy: 0.9976 - loss: 0.0360 - val_accuracy: 0.9827 - val_loss: 0.0716
Epoch 68/80
469/469 ————— 4s 8ms/step - accuracy: 0.9981 - loss: 0.0352 - val_accuracy: 0.9825 - val_loss: 0.0716
Epoch 69/80
469/469 ————— 4s 8ms/step - accuracy: 0.9975 - loss: 0.0360 - val_accuracy: 0.9830 - val_loss: 0.0717
Epoch 70/80
469/469 ————— 3s 7ms/step - accuracy: 0.9981 - loss: 0.0348 - val_accuracy: 0.9830 - val_loss: 0.0718
Epoch 71/80
469/469 ————— 3s 6ms/step - accuracy: 0.9974 - loss: 0.0359 - val_accuracy: 0.9831 - val_loss: 0.0715
Epoch 72/80
469/469 ————— 3s 6ms/step - accuracy: 0.9976 - loss: 0.0357 - val_accuracy: 0.9832 - val_loss: 0.0715
Epoch 73/80
469/469 ————— 3s 6ms/step - accuracy: 0.9979 - loss: 0.0350 - val_accuracy: 0.9827 - val_loss: 0.0715
Epoch 74/80
469/469 ————— 4s 8ms/step - accuracy: 0.9978 - loss: 0.0354 - val_accuracy: 0.9828 - val_loss: 0.0715
Epoch 75/80
469/469 ————— 3s 6ms/step - accuracy: 0.9977 - loss: 0.0355 - val_accuracy: 0.9827 - val_loss: 0.0715
Epoch 76/80
469/469 ————— 3s 6ms/step - accuracy: 0.9977 - loss: 0.0350 - val_accuracy: 0.9826 - val_loss: 0.0716

Epoch 77/80

469/469  **3s** 7ms/step - accuracy: 0.9979 - loss: 0.0349 - val_accuracy: 0.9825 - val_loss: 0.0714

Epoch 78/80

469/469  **4s** 8ms/step - accuracy: 0.9980 - loss: 0.0349 - val_accuracy: 0.9826 - val_loss: 0.0713

Epoch 79/80

469/469  **4s** 8ms/step - accuracy: 0.9976 - loss: 0.0353 - val_accuracy: 0.9828 - val_loss: 0.0712

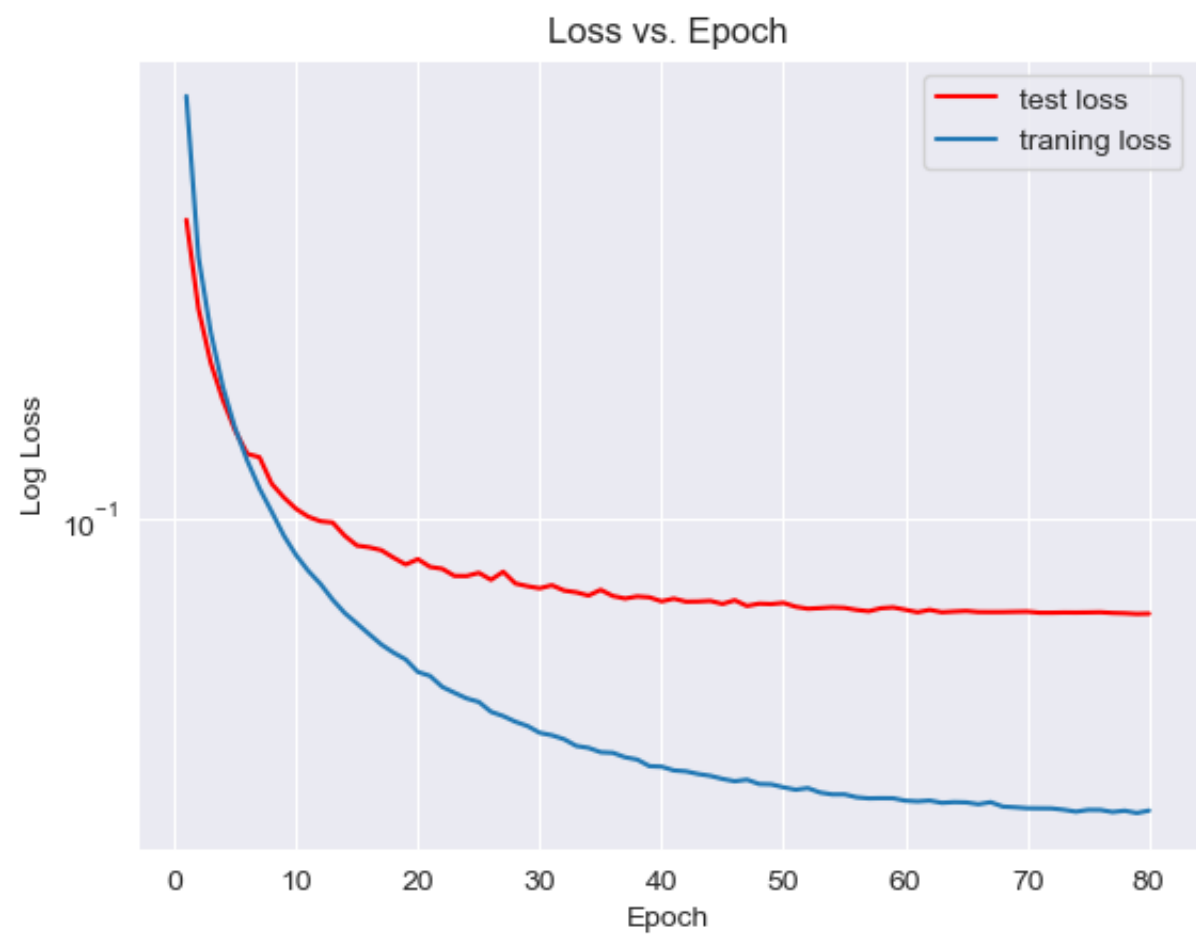
Epoch 80/80

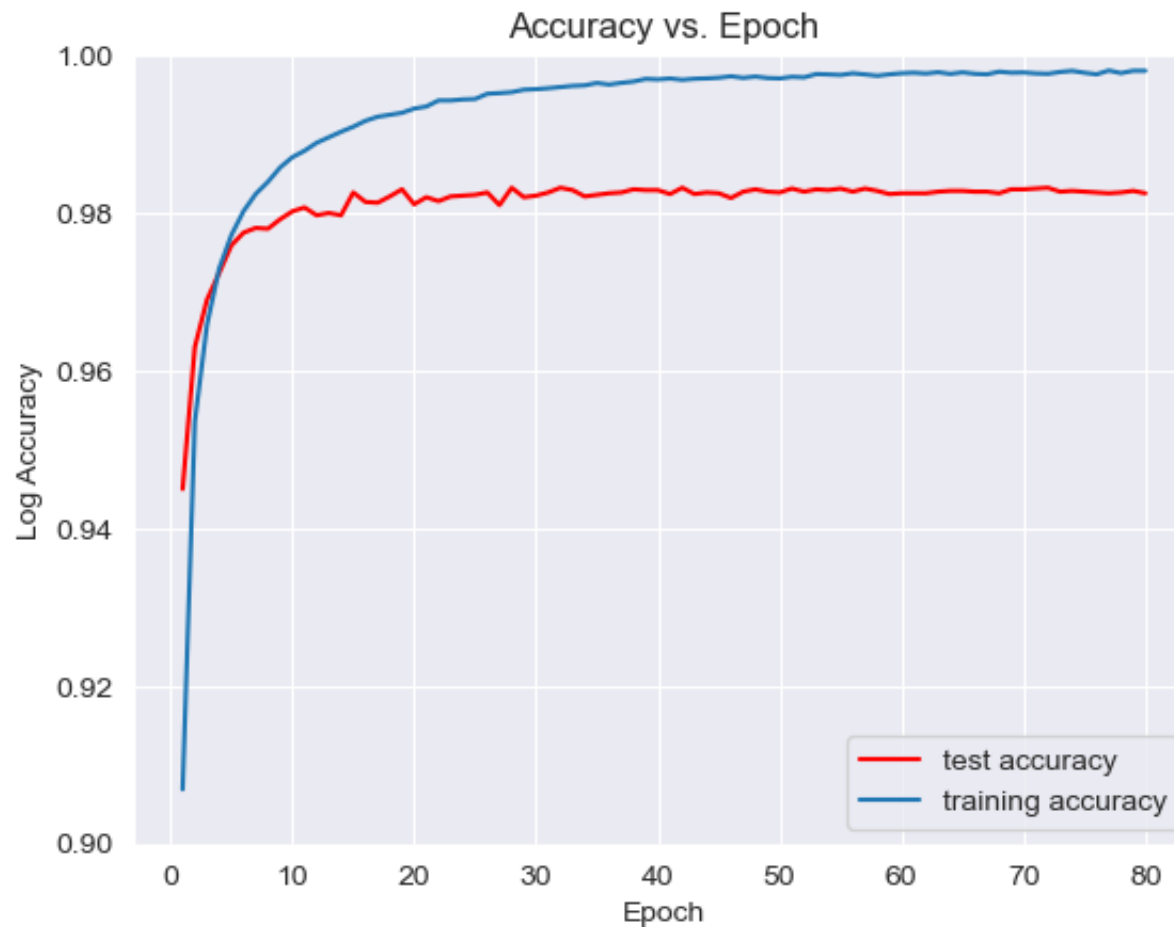
469/469  **3s** 7ms/step - accuracy: 0.9982 - loss: 0.0355 - val_accuracy: 0.9825 - val_loss: 0.0712

313/313  **1s** 2ms/step - accuracy: 0.9789 - loss: 0.0823

Test accuracy = 0.983 Test loss = 0.071

```
In [30]: plot_loss(history_nndo)
         plot_accuracy(history_nndo)
```





```
In [31]: test_loss, test_accuracy = nndo.evaluate(test_images, test_labels)
print("Test accuracy = {0:.3f}    Test loss = {1:.3f}".format(test_accuracy, test_loss))
```

313/313 ————— 1s 2ms/step – accuracy: 0.9789 – loss: 0.0823
 Test accuracy = 0.983 Test loss = 0.071

? Examine the accuracy value achieved and the curves for the training and validation metrics and answer these questions.

1. How do the final trajectory of the validation accuracy curve and validation accuracy compare to the previously fit model using batch normalization?

2. Examine the change in slope in this curve, comparing it to the perviously fit model using batch normalizatuion. What might the difference in the shapes tell you about the learning of this new model?

 Answers:

1. The final trajectories of the validation accuracy curve are similar, but the droupout regularized model achieves 98.3% which is even higher performance.
2. The new model stabilize in maximum accuracy area alsmost twice faster than the previous one. Should we use an early stopping, the dropout model would conclude its training around epoch 20 VS previous model that required almost 40 epochs to stabilize on its asymptotic performance level.

Copyright 2018, 2019, 2020, 2021, 2022, 2023, 2024, Stephen F Elston. All rights reserved

In []: